🔧 Jenkins Pipelines — Core: Complete Interview Mastery Guide

Category 3 | DevOps/SRE/Platform Engineer Interview Prep

> **How to use this guide:** Every topic follows the full teaching structure: Simple Definition → Why it exists → How it works → Key concepts → Short interview answer → Deep dive → Real-world example → Interview Q&A → Gotchas → Connections.
>
> ⚠ = Frequently misunderstood or heavily tested. Pay extra attention.
>
> This is the **most heavily tested category in Jenkins interviews**. Interviewers at senior level expect you to write pipeline code live, explain syntax choices, debug broken pipelines, and design pipeline architectures. Every code block here is real, production-grade Jenkinsfile syntax — study it until you can write it from memory.

📜 TABLE OF CONTENTS

1. Topic 3.1 — Freestyle Jobs vs Pipeline Jobs
2. Topic 3.2 — Declarative Pipeline — Full Syntax & Directives
3. Topic 3.3 — Scripted Pipeline — Groovy-Based Flexibility
4. Topic 3.4 — Declarative vs Scripted ⚠
5. Topic 3.5 — Jenkinsfile: Anatomy, Best Practices, Version Control
6. Topic 3.6 — Stages, Steps, and Post Actions
7. Topic 3.7 — Parallel Stages ⚠
8. Topic 3.8 — Pipeline Parameters
9. Topic 3.9 — Pipeline Triggers
10. Topic 3.10 — Stash and Unstash ⚠
11. Topic 3.11 — Environment Variables
12. Topic 3.12 — Credentials Management
13. Topic 3.13 — Pipeline Shared Libraries ⚠
14. Topic 3.14 — Matrix Builds
15. Category 3 Summary & Self-Quiz

Topic 3.1 — Freestyle Jobs vs Pipeline Jobs

🟢 Beginner | Historical Context + Modern Decision

📌 What It Is — In Simple Terms

Jenkins has two fundamental job types:

Freestyle Job — the original Jenkins job type. A GUI-configured job where you click through screens to define SCM, build triggers, build steps (shell commands, Maven goals), and post-build actions. Everything lives in Jenkins's database, not in version control.

Pipeline Job — the modern Jenkins job type. The build logic is defined in a `Jenkinsfile` using Groovy-based DSL, committed to version control alongside the application code. Executed as a structured pipeline with named stages.

```
Freestyle Job:
  Jenkins GUI → click → click → click → build
  Config stored in: JENKINS_HOME/jobs/<name>/config.xml
  Not version controlled. Not reviewable. Not reproducible.

Pipeline Job:
  Jenkinsfile in repo → Jenkins reads it → executes pipeline
  Config stored in: your Git repository
  Version controlled. Reviewable in PRs. Reproducible.
```

🔍 Why Pipeline Jobs Exist — The Problems Freestyle Couldn't Solve

Freestyle Problem	Pipeline Solution
Config only in Jenkins GUI — no version control	Jenkinsfile in Git — full history, PR review
No branching logic (if/else)	Full Groovy conditionals
No parallelism	`parallel {}` stages
No pipeline visualization	Stage view, Blue Ocean visualization
Can't model multi-step deployment flows	Stages with gates, approvals, environments
Can't easily retry specific stages	`retry()` and `input` steps
Secrets scattered, inconsistent	`withCredentials {}` binding
New project = manual GUI setup	Multibranch auto-discovers Jenkinsfiles
Can't test the build config	Pipeline unit testing with `jenkins-pipeline-unit`

⚙️ How Each Works

Freestyle Job — Under the Hood

```
<!-- JENKINS_HOME/jobs/my-freestyle-job/config.xml (generated by GUI) -->
<?xml version='1.1' encoding='UTF-8'?>
<project>
  <scm class="hudson.plugins.git.GitSCM">
```

```
    <userRemoteConfigs>
      <hudson.plugins.git.UserRemoteConfig>
        <url>https://github.com/myorg/myapp.git</url>
      </hudson.plugins.git.UserRemoteConfig>
    </userRemoteConfigs>
    <branches>
      <hudson.plugins.git.BranchSpec><name>*/main</name>
</hudson.plugins.git.BranchSpec>
    </branches>
  </scm>

  <triggers>
    <hudson.triggers.SCMTrigger>
      <spec>H/5 * * * *</spec>  <!-- Polling every 5 min -->
    </hudson.triggers.SCMTrigger>
  </triggers>

  <builders>
    <hudson.tasks.Shell>
      <command>mvn clean package</command>
    </hudson.tasks.Shell>
    <hudson.tasks.Shell>
      <command>docker build -t myapp:latest .</command>
    </hudson.tasks.Shell>
  </builders>

  <publishers>
    <hudson.tasks.Mailer>
      <recipients>dev-team@example.com</recipients>
    </hudson.tasks.Mailer>
  </publishers>
</project>
```

```
Execution model:
  1. Run ALL build steps sequentially in one shell session
  2. Post-build actions run at end
  3. No named stages — one flat sequence
  4. No visualization beyond pass/fail
  5. Failure at step 2 runs all remaining post-build actions
```

Pipeline Job — Under the Hood

```
// Jenkinsfile at root of repo
// Rich structured execution model

pipeline {
    agent { label 'linux' }

    stages {
```

```
        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }
        stage('Docker Build') {
            steps {
                sh 'docker build -t myapp:${GIT_COMMIT[0..6]} .'
            }
        }
    }

    post {
        failure {
            mail to: 'dev-team@example.com',
                 subject: "Build Failed: ${env.JOB_NAME}",
                 body: "Build URL: ${env.BUILD_URL}"
        }
    }
}
```

```
Execution model:
  1. Named stages with clear boundaries
  2. Stage view shows which stage failed and why
  3. post{} runs per-stage or globally with conditions (always, success,
failure)
  4. Config lives in Git — any change goes through PR
  5. Multibranch: every branch gets its own pipeline automatically
```

🔑 Key Concepts

Concept	Meaning
Freestyle Job	GUI-configured, sequential, config in Jenkins DB
Pipeline Job	Jenkinsfile-based, structured stages, config in Git
Multibranch Pipeline	Automatically creates a pipeline per branch using each branch's Jenkinsfile
Organization Folder	Scans entire GitHub org — creates pipelines for every repo with a Jenkinsfile
Stage View	Visual representation of pipeline stages and their status
Blue Ocean	Jenkins UI plugin with enhanced pipeline visualization
Job DSL	Plugin for generating Freestyle jobs programmatically (legacy approach)

💬 Short Crisp Interview Answer

> *"Freestyle jobs are the legacy Jenkins job type — configured entirely through the GUI, stored in Jenkins's internal XML database, not version controlled. They work for simple, single-step builds but don't scale. Pipeline jobs are the modern standard — the entire build definition is a Jenkinsfile committed to the repository. This gives you version control, PR review for pipeline changes, named stages with visualization, parallelism, conditional logic, proper failure handling, and automatic discovery via Multibranch pipelines. In any production environment I'd build today, Freestyle jobs have no place — everything is Jenkinsfiles in source control."*

🔬 Deep Dive Answer

When Freestyle Jobs Are Still Legitimate:

Despite being legacy, there are valid reasons to encounter or maintain Freestyle jobs:

1. **Trigger orchestration** — A Freestyle job that only triggers downstream Pipeline jobs, with no build logic itself. Some teams use this as a "trigger board."
2. **Legacy migration in progress** — A team actively migrating from Freestyle to Pipeline — keep the Freestyle job running while the Jenkinsfile is developed and tested.
3. **Simple one-step utility jobs** — Running a daily database backup script with no branching logic, no stages needed. The overhead of a Jenkinsfile is unjustified.
4. **Job DSL-generated jobs** — In organizations using the Job DSL plugin heavily for Freestyle job generation before Multibranch existed.

The Job DSL Plugin (Freestyle at Scale — now largely superseded):

```
// Job DSL — programmatically create Freestyle jobs
// Runs in a "seed job" that generates other jobs

job('myapp-build') {
    scm {
        git('https://github.com/myorg/myapp.git') {
            branch('*/main')
        }
    }
    triggers { scm('H/5 * * * *') }
    steps {
        shell('mvn clean package')
        shell('docker build -t myapp:latest .')
    }
    publishers {
        mailer('dev@example.com', true, true)
    }
}
// Job DSL is the "Pipeline as Code for Freestyle jobs"
// Largely replaced by Multibranch Pipeline + Jenkinsfile in modern
Jenkins
```

🏭 Real-World Production Example

BBC's Jenkins Migration: BBC News migrated 400 Freestyle jobs to Pipeline jobs over 18 months. The trigger: a Jenkins upgrade broke 40 Freestyle jobs because undocumented GUI configurations didn't survive the upgrade. After migration, every pipeline configuration change went through a PR review, reducing "surprise broken builds" incidents by 70%. The Multibranch pipeline auto-discovery eliminated manual job creation for each new feature branch — developers saw their feature branch pipelines appear automatically within minutes of pushing.

❓ Common Interview Questions & Strong Answers

Q1: Would you ever use a Freestyle job in a new project today?

> Almost never. The only cases I'd consider: a very simple, single-step utility job with no branching logic, or as a trigger relay. Everything involving a real build, test, and deploy cycle gets a Jenkinsfile. The advantages of Pipeline — version control, reviewability, visualization, parallelism, proper credential handling — outweigh any simplicity benefit of Freestyle. Even a "simple" Freestyle job becomes hard to maintain when you need to add a second step, retry logic, or environment-specific behavior.

Q2: What's the difference between a Pipeline job, a Multibranch Pipeline, and an Organization Folder?

> A Pipeline job runs a single, manually specified Jenkinsfile — you configure which repo and branch. A Multibranch Pipeline scans a single repository and automatically creates a pipeline for every branch that contains a Jenkinsfile — feature branches, PRs, main — each gets its own build history. An Organization Folder scans an entire GitHub organization or GitLab group and creates a Multibranch Pipeline for every repository that has a Jenkinsfile. Organization Folders are the zero-maintenance option — new repos automatically get pipelines without any Jenkins administrator involvement.

⚠ Tricky Edge Cases & Gotchas

- **Freestyle jobs have no stage visualization.** When a Freestyle job fails, you get one blob of console output. With Pipeline stages, you see exactly which stage failed, how long each stage took, and can re-run specific stages. This operational visibility difference is significant in production.
- **Freestyle + downstream triggering creates invisible dependencies.** Complex Freestyle job chains (job A triggers job B triggers job C) create implicit dependencies that aren't visible or versioned. Pipeline makes these explicit with stages, `build` steps, and `parallel` blocks.
- **Job DSL vs Jenkinsfile confusion.** Job DSL generates job configuration programmatically — it's NOT the same as a Jenkinsfile. Job DSL creates Freestyle (or Pipeline wrapper) jobs. A Jenkinsfile IS the Pipeline definition. Interviewers sometimes ask about this distinction.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Declarative Pipeline (3.2)	The Jenkinsfile syntax for Pipeline jobs
Jenkinsfile (3.5)	The file that replaces Freestyle GUI configuration

Connected Topic	How It Connects
Multi-Branch Pipelines (4.1)	Auto-discovers Jenkinsfiles across branches
Pipeline as Code (1.6)	The philosophy behind why Pipeline > Freestyle
JCasC (5.7)	Can define both Freestyle and Pipeline jobs as code

---

Topic 3.2 — Declarative Pipeline — Full Syntax and Directives

🟢 Beginner | The Modern Standard

📌 What It Is — In Simple Terms

Declarative Pipeline is the recommended, structured syntax for writing Jenkins pipelines. Introduced in Jenkins 2.x, it wraps pipeline logic in a strict `pipeline { }` block with well-defined sections (`agent`, `stages`, `environment`, `options`, `post`, etc.). It provides a clean, readable, opinionated structure that is easier to learn, read, and maintain than Scripted Pipeline.

Think of it as a structured form with labeled fields — you fill in the right sections, and Jenkins handles the rest.

⚙️ The Complete Declarative Pipeline Anatomy

```
// COMPLETE DECLARATIVE PIPELINE — every directive shown
// This is the reference template. In practice, use only what you need.

pipeline {

    // ── AGENT ──────────────────────────────────────────────────────
    // WHERE the pipeline runs. Required at top level.
    agent {
        // Option 1: Any available agent
        any

        // Option 2: Specific label
        // label 'linux && docker'

        // Option 3: No global agent (define per-stage)
        // none

        // Option 4: Docker container as agent
        // docker {
        //     image 'maven:3.9-eclipse-temurin-17'
        //     args  '-v /root/.m2:/root/.m2'
        //     label 'docker'
```

```
        // }

        // Option 5: Kubernetes pod
        // kubernetes {
        //     yaml '''...pod spec...'''
        // }
    }

    // ── OPTIONS ──────────────────────────────────────────────────────
    // Pipeline-wide behavior settings
    options {
        // Build retention - keep last 10 builds, artifacts for 5
        buildDiscarder(logRotator(numToKeepStr: '10',
artifactNumToKeepStr: '5'))

        // Timeout the ENTIRE pipeline after 1 hour
        timeout(time: 1, unit: 'HOURS')

        // Don't run concurrent builds of this pipeline
        disableConcurrentBuilds()

        // Add timestamps to every log line
        timestamps()

        // Colorize ANSI output (requires AnsiColor plugin)
        ansiColor('xterm')

        // Skip the automatic SCM checkout in the first stage
        // Useful when you want to control checkout manually
        skipDefaultCheckout(true)

        // Pipeline durability hint - performance vs recoverability
        durabilityHint('PERFORMANCE_OPTIMIZED')

        // Retry the whole pipeline up to 2 times on failure
        // (usually too aggressive - prefer per-stage retry)
        // retry(2)

        // Don't show changelog in build description
        skipStagesAfterUnstable()

        // Quiet period - wait N seconds before starting
        quietPeriod(10)
    }

    // ── ENVIRONMENT ──────────────────────────────────────────────────
    // Environment variables available to ALL stages
    environment {
        // Static values
        APP_NAME     = 'myservice'
        REGISTRY     = 'registry.example.com'
        IMAGE_TAG    =
"${APP_NAME}:${env.BUILD_NUMBER}-${env.GIT_COMMIT[0..6]}"
```

```
        // From Jenkins credentials store — injected securely
        DOCKER_CREDS = credentials('dockerhub-creds')
        // Creates: DOCKER_CREDS_USR (username) and DOCKER_CREDS_PSW
(password)

        // String secret
        SONAR_TOKEN  = credentials('sonarqube-token')

        // Dynamic value using shell command
        VERSION      = sh(script: 'cat VERSION', returnStdout:
true).trim()
    }

    // ── PARAMETERS ──────────────────────────────────────────────
    // Build parameters (defined once, available as params.NAME)
    parameters {
        string(
            name:         'TARGET_ENV',
            defaultValue: 'staging',
            description:  'Environment to deploy to'
        )
        choice(
            name:    'LOG_LEVEL',
            choices: ['INFO', 'DEBUG', 'WARN', 'ERROR'],
            description: 'Log level for the deployed service'
        )
        booleanParam(
            name:         'SKIP_TESTS',
            defaultValue: false,
            description:  'Skip test execution (use for hotfix deploys
only)'
        )
        password(
            name:         'DEPLOY_TOKEN',
            defaultValue: '',
            description:  'Deployment authorization token'
        )
    }

    // ── TRIGGERS ────────────────────────────────────────────────
    // How the pipeline is triggered (in addition to manual)
    triggers {
        // Cron schedule
        cron('H 2 * * 1-5')         // 2am weekdays

        // Poll SCM every 5 minutes (avoid — use webhooks instead)
        // pollSCM('H/5 * * * *')

        // GitHub webhook (install GitHub plugin)
        githubPush()

        // Trigger when upstream job succeeds
        upstream(upstreamProjects: 'myapp-build-pipeline', threshold:
'SUCCESS')
```

```
    }

    // ── TOOLS ──────────────────────────────────────────────────────────
    // Pre-install tools from Global Tool Configuration
    // Tool names must match exactly what's configured in Manage Jenkins →
Global Tools
    tools {
        maven 'Maven-3.9'          // Installs Maven, adds mvn to PATH
        jdk   'JDK-17'             // Installs JDK, sets JAVA_HOME
        // nodejs 'NodeJS-18'      // Requires NodeJS plugin
        // gradle 'Gradle-8.5'     // Requires Gradle plugin
    }

    // ── STAGES ─────────────────────────────────────────────────────────
    // The core pipeline logic — sequence of named stages
    stages {

        // ── SIMPLE STAGE ───────────────────────────────────────────────
        stage('Checkout') {
            steps {
                // Explicit checkout (when skipDefaultCheckout is true)
                checkout([
                    $class: 'GitSCM',
                    branches: [[name: env.BRANCH_NAME]],
                    extensions: [
                        [$class: 'CleanBeforeCheckout'],
                        [$class: 'CloneOption', shallow: true, depth: 1]
                    ],
                    userRemoteConfigs: [[
                        url: 'https://github.com/myorg/myapp.git',
                        credentialsId: 'github-creds'
                    ]]
                ])
            }
        }

        // ── STAGE WITH WHEN CONDITION ──────────────────────────────────
        stage('Run Tests') {
            when {
                // Only run when NOT skipping tests
                not { expression { return params.SKIP_TESTS } }
            }
            steps {
                sh 'mvn test'
            }
            post {
                always {
                    junit 'target/surefire-reports/*.xml'
                    publishCoverage(adapters:
[jacocoAdapter('target/site/jacoco/jacoco.xml')])
                }
            }
        }
```

```
        // ── STAGE WITH PARALLEL SUBSTAGES ──────────────────────────────
        stage('Quality Gates') {
            parallel {
                stage('SAST Scan') {
                    steps { sh 'semgrep --config auto src/' }
                }
                stage('Dependency Check') {
                    steps { sh 'mvn dependency-check:check' }
                }
                stage('SonarQube') {
                    steps {
                        withSonarQubeEnv('SonarQube-Server') {
                            sh 'mvn sonar:sonar'
                        }
                        timeout(time: 10, unit: 'MINUTES') {
                            waitForQualityGate abortPipeline: true
                        }
                    }
                }
            }
        }

        // ── STAGE WITH NESTED STAGES (sequential group) ────────────────
        stage('Build & Package') {
            stages {
                stage('Compile') {
                    steps { sh 'mvn clean package -DskipTests' }
                }
                stage('Build Docker Image') {
                    steps {
                        sh "docker build -t ${IMAGE_TAG} ."
                        sh "trivy image --exit-code 1 --severity
HIGH,CRITICAL ${IMAGE_TAG}"
                    }
                }
                stage('Push to Registry') {
                    steps {
                        sh "echo ${DOCKER_CREDS_PSW} | docker login
${REGISTRY} -u ${DOCKER_CREDS_USR} --password-stdin"
                        sh "docker push ${IMAGE_TAG}"
                    }
                }
            }
        }

        // ── STAGE WITH ENVIRONMENT OVERRIDE ────────────────────────────
        stage('Deploy to Staging') {
            environment {
                // This env var exists ONLY within this stage
                DEPLOY_ENV  = 'staging'
                REPLICA_COUNT = '2'
            }
            steps {
                sh """
```

```
                    helm upgrade --install ${APP_NAME} ./chart \
                        --namespace ${DEPLOY_ENV} \
                        --set
image.tag=${env.BUILD_NUMBER}-${env.GIT_COMMIT[0..6]} \
                        --set replicaCount=${REPLICA_COUNT} \
                        -f values-${DEPLOY_ENV}.yaml \
                        --wait --timeout=5m
                """
            }
        }

        // ── STAGE WITH INPUT (manual approval gate) ──────────────────
        stage('Approve Production Deploy') {
            when {
                branch 'main'  // Only require approval on main branch
            }
            steps {
                input(
                    message: "Deploy ${IMAGE_TAG} to production?",
                    ok:      'Deploy',
                    submitter: 'release-managers,ops-team',
                    parameters: [
                        choice(
                            name: 'STRATEGY',
                            choices: ['canary', 'blue-green', 'rolling'],
                            description: 'Deployment strategy'
                        )
                    ]
                )
            }
        }

        // ── STAGE WITH RETRY AND TIMEOUT ─────────────────────────────
        stage('Deploy to Production') {
            when { branch 'main' }
            options {
                timeout(time: 10, unit: 'MINUTES')
                retry(2)
            }
            steps {
                sh """
                    helm upgrade --install ${APP_NAME} ./chart \
                        --namespace production \
                        --set
image.tag=${env.BUILD_NUMBER}-${env.GIT_COMMIT[0..6]} \
                        -f values-production.yaml \
                        --wait --timeout=8m
                """
            }
        }

        // ── STAGE WITH SCRIPT BLOCK (Scripted inside Declarative) ────
        stage('Post-Deploy Validation') {
            steps {
```

```
                script {
                    // Use Groovy logic inside Declarative pipeline
                    def maxRetries = 10
                    def sleepSeconds = 30
                    def healthy = false

                    for (int i = 0; i < maxRetries; i++) {
                        def status = sh(
                            script: "curl -s -o /dev/null -w '%
{http_code}' https://production.example.com/health",
                            returnStdout: true
                        ).trim()

                        if (status == '200') {
                            healthy = true
                            echo "✅ Health check passed after ${i+1}
attempt(s)"
                            break
                        }
                        echo "⏳ Health check attempt
${i+1}/${maxRetries}: status=${status}, retrying in ${sleepSeconds}s"
                        sleep(sleepSeconds)
                    }

                    if (!healthy) {
                        error("❌ Service failed health checks after
${maxRetries} attempts — triggering rollback")
                    }
                }
            }
        }
    }

    // ── POST ──────────────────────────────────────────────────────────
    // Actions to run after ALL stages complete
    post {
        always {
            // Always runs — cleanup, notifications regardless of result
            cleanWs()
            script {
                def duration = currentBuild.durationString.replace(' and
counting', '')
                echo "Pipeline completed in: ${duration}"
            }
        }

        success {
            slackSend(
                channel:  '#deployments',
                color:    'good',
                message:  "✅ *${env.JOB_NAME}* #${env.BUILD_NUMBER}
succeeded (${currentBuild.durationString})\n${env.BUILD_URL}"
            )
        }
```

```
        failure {
            slackSend(
                channel:  '#build-failures',
                color:    'danger',
                message:  "❌ *${env.JOB_NAME}* #${env.BUILD_NUMBER}
FAILED at stage: ${env.STAGE_NAME}\n${env.BUILD_URL}"
            )
            // Email to committer
            emailext(
                subject: "Build Failed: ${env.JOB_NAME}
#${env.BUILD_NUMBER}",
                body:    "See build: ${env.BUILD_URL}",
                to:      "\${DEFAULT_RECIPIENTS}",
                recipientProviders: [culprits(), requestor()]
            )
        }

        unstable {
            // Build completed but some tests failed (UNSTABLE state)
            slackSend(
                channel: '#build-failures',
                color:   'warning',
                message: "⚠ *${env.JOB_NAME}* #${env.BUILD_NUMBER}
UNSTABLE - test failures\n${env.BUILD_URL}"
            )
        }

        aborted {
            echo "Pipeline was manually aborted"
        }

        changed {
            // Only runs if status changed from last build (e.g.,
failure→success)
            slackSend(
                channel: '#deployments',
                message: "🔄 Build status changed:
${currentBuild.currentResult}"
            )
        }

        fixed {
            // Status changed from failure to success
            slackSend(
                channel: '#deployments',
                color:   'good',
                message: "🎉 Build FIXED: ${env.JOB_NAME}"
            )
        }

        regression {
            // Status changed from success to failure
            slackSend(
```

```
                channel: '#build-failures',
                color:   'danger',
                message: "📉 Build REGRESSED: ${env.JOB_NAME}"
            )
        }
    }
}
```

🔑 Directive Reference

Directive	Location	Purpose
`agent`	pipeline / stage	Where to run
`options`	pipeline / stage	Behavior settings (timeout, retry, discard)
`environment`	pipeline / stage	Environment variables
`parameters`	pipeline	Build parameters
`triggers`	pipeline	Automatic trigger conditions
`tools`	pipeline / stage	Pre-install global tools
`stages`	pipeline	Container for all stage blocks
`stage`	inside stages	Named execution unit
`steps`	inside stage	Actual build commands
`parallel`	inside stage	Parallel sub-stages
`when`	inside stage	Conditional execution
`post`	pipeline / stage	Cleanup and notifications
`input`	inside steps	Manual approval gate
`script`	inside steps	Groovy scripting block

The `when` Directive — All Conditions

```
stage('Deploy') {
    when {
        // Branch name matches pattern
        branch 'main'
        branch pattern: 'release/.*', comparator: 'REGEXP'

        // Environment variable condition
        environment name: 'TARGET_ENV', value: 'production'

        // Boolean expression
        expression { return params.DEPLOY_TO_PROD == true }
```

```
        // NOT condition
        not { branch 'feature/*' }

        // ANY of these conditions (OR)
        anyOf {
            branch 'main'
            branch 'release/*'
        }

        // ALL of these conditions (AND)
        allOf {
            branch 'main'
            environment name: 'CI', value: 'true'
        }

        // Changed files (requires changedRequest or specific SCM)
        changeset 'src/**'
        changeset pattern: '.*\\.java', comparator: 'REGEXP'

        // Build is triggered by a specific user
        triggeredBy 'UserIdCause'

        // Tag build
        tag 'v*'

        // Pull Request
        changeRequest()
        changeRequest target: 'main'
    }
    steps {
        sh 'helm upgrade ...'
    }
}
```

💬 Short Crisp Interview Answer

> *"Declarative Pipeline is the recommended Jenkins pipeline syntax, introduced with Jenkins 2.x. It wraps all pipeline logic in a `pipeline {}` block with a fixed set of sections: `agent` defines where it runs, `environment` sets variables, `options` controls pipeline behavior, `parameters` defines input parameters, `triggers` configures automatic triggers, `stages` contains the named build stages, and `post` defines cleanup and notification actions per build result. The key advantage of Declarative over Scripted is its structure — it's opinionated, validated before execution, easier to read for non-Groovy developers, and most IDE plugins provide syntax completion. You escape to full Groovy scripting when needed via the `script {}` block inside any `steps {}` block."*

🔬 Deep Dive Answer

Declarative Pipeline Validation:

```
# Validate Jenkinsfile syntax before committing:
# Method 1: Jenkins Declarative Linter (via API)
curl -X POST \
  -F "jenkinsfile=<Jenkinsfile" \
  https://jenkins.example.com/pipeline-model-converter/validate \
  -u admin:${TOKEN}
# Returns: Jenkinsfile successfully validated. OR error details

# Method 2: Jenkins CLI
java -jar jenkins-cli.jar \
  -s https://jenkins.example.com \
  -auth admin:${TOKEN} \
  declarative-linter < Jenkinsfile

# Method 3: Jenkins Replay button (in UI)
# Lets you edit and test Jenkinsfile changes without committing
```

Stage-Level vs Pipeline-Level Directives:

```
// Some directives work at BOTH pipeline level AND stage level
// Stage-level directives OVERRIDE pipeline-level for that stage

pipeline {
    agent any
    options {
        timeout(time: 1, unit: 'HOURS')  // Pipeline: 1 hour total
    }

    stages {
        stage('Quick Test') {
            options {
                timeout(time: 5, unit: 'MINUTES')  // Stage: 5 min max
                retry(3)                            // Retry this stage
only
            }
            steps { sh 'mvn test' }
        }

        stage('Deploy') {
            agent { label 'deploy-agent' }  // Different agent for this
stage
            environment {
                DEPLOY_KEY = credentials('deploy-ssh-key')  // Stage-local
credential
            }
            steps { sh 'ansible-playbook deploy.yml' }
        }
    }
}
```

Post Actions at Stage Level:

```
stage('Test') {
    steps {
        sh 'mvn test'
    }
    post {
        // Stage-level post - runs AFTER THIS STAGE, not the whole
pipeline
        always {
            junit 'target/surefire-reports/*.xml'
        }
        failure {
            // Archive test results even on failure for debugging
            archiveArtifacts artifacts: 'target/surefire-reports/**',
                             allowEmptyArchive: true
        }
    }
}
```

🏭 Real-World Production Example

Thoughtworks' Standard Pipeline Template: Thoughtworks published their "Build Your Own Jenkins Pipeline" standard using Declarative Pipeline. Their corporate template uses every directive: `options { buildDiscarder, timeout, timestamps }`, `environment` for image tags and registry URLs, `parameters` for environment targeting, `stages` with `when` conditions per environment, and `post` with Slack notifications. The key discipline: the template is version-controlled in a central repository and shared via Shared Libraries — no team invents their own pipeline structure from scratch.

❓ Common Interview Questions & Strong Answers

Q1: What is the `script {}` block in Declarative Pipeline and when do you use it?

> The `script {}` block allows you to write arbitrary Groovy code inside a Declarative Pipeline `steps {}` block. Declarative Pipeline's structured directives cover 90% of use cases, but sometimes you need: dynamic variable assignment, loops, conditionals beyond `when {}`, calling methods, map operations, or error handling logic. The `script {}` block gives you full Groovy access at the cost of losing some Declarative validations for that block. Best practice: keep `script {}` blocks small and focused. If a `script {}` block is getting large, it's a signal to extract logic into a Shared Library function.

Q2: How do the `post` conditions differ? When is `always` vs `success` vs `changed` used?

> `always` runs unconditionally — use for cleanup (`cleanWs()`), final notifications, or publishing test results regardless of outcome. `success` only runs on green builds — use for Slack success messages, deployment notifications. `failure` only on failures — use for alerting, paging. `unstable` only on builds with test failures — use for test failure notifications. `changed` runs only when the build

> result is DIFFERENT from the last build — use to notify "the build was fixed" or "the build broke." `fixed` is specifically success after failure; `regression` is failure after success. In practice, I use `always` + `success` + `failure` for most pipelines, and add `changed` for teams that want "the build is fixed" notifications without constant success noise.

⚠ Tricky Edge Cases & Gotchas

- ⚠ **`environment {}` credential binding creates TWO variables.** `credentials('my-userpass-cred')` creates `VAR_USR` and `VAR_PSW` (username and password separately). Many developers expect just `VAR` and are confused why it doesn't work. The combined value is also set as `VAR=username:password`.
- **`when` conditions evaluated BEFORE stage agent allocation.** If you have `agent none` at pipeline level and per-stage agents, `when` is evaluated before the stage agent spins up. This is correct and expected but surprises people who expect the stage environment to be available when `when` evaluates.
- **`parameters` block only applied after FIRST manual build.** When you add a `parameters {}` block to a Jenkinsfile for the first time and push it, the first build triggered by the push won't use those parameters — Jenkins hasn't seen the new Jenkinsfile yet. The first build discovers the parameters, and subsequent builds can use them.
- **`options { retry(N) }` at pipeline level retries ALL stages.** This is usually too aggressive. If stage 3 fails and you retry the whole pipeline, stages 1 and 2 run again unnecessarily. Prefer `options { retry(N) }` at stage level for the specific stage that needs retry.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Scripted Pipeline (3.3)	Alternative syntax — know both and when to use each
Stages/Steps/Post (3.6)	Core building blocks of Declarative syntax
Parallel Stages (3.7)	`parallel {}` block within Declarative
Parameters (3.8)	`parameters {}` directive
Triggers (3.9)	`triggers {}` directive
Environment Variables (3.11)	`environment {}` directive
Credentials (3.12)	`credentials()` inside `environment {}`
Shared Libraries (3.13)	Libraries provide reusable steps called within Declarative

---

Topic 3.3 — Scripted Pipeline — Groovy-Based Flexibility

🟡 Intermediate | The Original Power Model

📌 What It Is — In Simple Terms

Scripted Pipeline is the original Jenkins Pipeline syntax, using pure Groovy within `node { }` blocks. It gives maximum flexibility — any Groovy language construct is valid — at the cost of less structure and no built-in validation. The developer is responsible for structuring the pipeline correctly.

Think of it as: Declarative = form with labeled fields; Scripted = blank sheet of Groovy paper.

⚙️ The Complete Scripted Pipeline Syntax

```
// SCRIPTED PIPELINE — complete reference

// Required: node block — allocates an executor on matching agent
node('linux && docker') {   // Label expression in quotes

    // Environment variables (must be set explicitly — no environment{}
block)
    def APP_NAME  = 'myservice'
    def REGISTRY  = 'registry.example.com'
    def IMAGE_TAG = "${APP_NAME}:${BUILD_NUMBER}-${env.GIT_COMMIT[0..6]}"

    // Try-catch-finally for error handling
    try {

        // ── STAGE 1: CHECKOUT ──────────────────────────────────────
        stage('Checkout') {
            checkout scm   // Uses job's configured SCM
            // OR explicit:
            // git branch: 'main',
            //     credentialsId: 'github-creds',
            //     url: 'https://github.com/myorg/myapp.git'
        }

        // ── STAGE 2: BUILD ─────────────────────────────────────────
        stage('Build') {
            // Groovy variables, methods, loops — all valid
            def mvnHome = tool name: 'Maven-3.9', type:
'hudson.tasks.Maven$MavenInstallation'
            env.PATH = "${mvnHome}/bin:${env.PATH}"

            sh 'mvn clean package -DskipTests'

            // Capture output from shell command
            def gitLog = sh(script: 'git log --oneline -5', returnStdout:
true).trim()
            echo "Recent commits:\n${gitLog}"
        }

        // ── STAGE 3: TEST (with Groovy loops) ──────────────────────
        stage('Test') {
```

```
            // Scripted allows real Groovy loops
            def testSuites = ['unit', 'integration', 'contract']
            def testResults = [:]

            testSuites.each { suite ->
                stage("Test: ${suite}") {  // Dynamic stage names!
                    try {
                        sh "mvn test -Psuite-${suite}"
                        testResults[suite] = 'PASSED'
                    } catch (Exception e) {
                        testResults[suite] = 'FAILED'
                        // Don't fail immediately - collect all results
                        currentBuild.result = 'UNSTABLE'
                    } finally {
                        junit "target/${suite}-reports/*.xml"
                    }
                }
            }

            // Evaluate collected results
            testResults.each { suite, result ->
                echo "Suite [${suite}]: ${result}"
            }
        }

        // ── STAGE 4: PARALLEL BUILDS ──────────────────────────────
        stage('Build Docker Images') {
            // Scripted parallel with dynamic map
            def services   = ['auth-service', 'payment-service', 'order-
service']
            def buildSteps = [:]

            services.each { service ->
                def svc = service  // Capture for closure - CRITICAL
gotcha!
                buildSteps[svc] = {
                    node('docker') {
                        sh "docker build -t
${REGISTRY}/${svc}:${BUILD_NUMBER} services/${svc}"
                        sh "docker push
${REGISTRY}/${svc}:${BUILD_NUMBER}"
                    }
                }
            }

            parallel buildSteps   // Launch all in parallel
        }

        // ── STAGE 5: DEPLOY WITH CONDITIONAL LOGIC ────────────────
        stage('Deploy') {
            // Full Groovy if/else/switch - not possible in Declarative
without script{}
            def targetEnv = env.BRANCH_NAME == 'main' ? 'production' :
                            env.BRANCH_NAME.startsWith('release/') ?
```

```
'staging' : 'dev'

            switch(targetEnv) {
                case 'production':
                    // Manual approval step
                    def approval = input(
                        message: "Deploy to PRODUCTION?",
                        ok: 'Deploy',
                        submitter: 'release-managers',
                        parameters: [
                            choice(name: 'CANARY_PERCENT',
                                   choices: ['5', '25', '50', '100'],
                                   description: 'Initial canary %')
                        ]
                    )
                    deployToEnvironment(targetEnv, IMAGE_TAG,
approval['CANARY_PERCENT'])
                    break
                case 'staging':
                    deployToEnvironment(targetEnv, IMAGE_TAG, '100')
                    break
                default:
                    deployToEnvironment(targetEnv, IMAGE_TAG, '100')
            }
        }

        // ── STAGE 6: NOTIFY SUCCESS ──────────────────────────────
        stage('Notify') {
            slackSend channel: '#deployments',
                      color:   'good',
                      message: "✅ ${APP_NAME} ${IMAGE_TAG} deployed"
        }

    } catch (Exception e) {
        // Catch any stage failure
        currentBuild.result = 'FAILURE'
        slackSend channel: '#build-failures',
                  color:   'danger',
                  message: "❌ Build FAILED:
${APP_NAME}\n${e.getMessage()}\n${BUILD_URL}"
        throw e  // Re-throw to mark build as failed in Jenkins

    } finally {
        // Always runs — equivalent of post { always {} } in Declarative
        cleanWs()
        echo "Build ${currentBuild.result ?: 'SUCCESS'} — workspace
cleaned"
    }
}

// ── HELPER FUNCTION ──────────────────────────────────────────────
// Functions can be defined outside node{} block
def deployToEnvironment(String env, String imageTag, String canaryPct) {
    echo "Deploying ${imageTag} to ${env} with ${canaryPct}% canary"
```

```
    sh """
        helm upgrade --install myservice ./chart \
            --namespace ${env} \
            --set image.tag=${imageTag.split(':')[1]} \
            --set canary.weight=${canaryPct} \
            -f values-${env}.yaml \
            --wait
    """
}
```

🔑 Scripted Pipeline Unique Capabilities

```
// 1. DYNAMIC STAGE NAMES (impossible in Declarative)
def environments = ['dev', 'qa', 'staging', 'production']
environments.each { env ->
    stage("Deploy to ${env}") {   // Stage name is a variable!
        sh "helm upgrade --install app ./chart --namespace ${env}"
    }
}

// 2. STAGES INSIDE LOOPS
def services = readFile('services.txt').readLines()
services.each { svc ->
    stage("Build ${svc}") {
        sh "docker build -t ${svc}:latest services/${svc}"
    }
}

// 3. COMPLEX ERROR HANDLING LOGIC
stage('Flaky Test') {
    def attempts = 0
    def maxAttempts = 3
    def passed = false

    while (attempts < maxAttempts && !passed) {
        try {
            sh 'pytest tests/ --reruns=0'
            passed = true
        } catch (Exception e) {
            attempts++
            if (attempts >= maxAttempts) throw e
            echo "Attempt ${attempts} failed, retrying..."
            sleep(10)
        }
    }
}

// 4. NODE SWITCHING MID-PIPELINE
node('linux') {
    stage('Build on Linux') {
```

```
        sh 'make build-linux'
        stash name: 'linux-binary', includes: 'dist/linux/**'
    }
}

node('windows') {
    stage('Build on Windows') {
        bat 'msbuild MyProject.sln /p:Configuration=Release'
        stash name: 'windows-binary', includes: 'dist/windows/**'
    }
}

node('macos') {
    stage('Build on macOS') {
        sh 'xcodebuild -scheme MyApp -configuration Release'
    }
}

// 5. CALLING SHARED LIBRARY FUNCTIONS IN SCRIPTED
@Library('jenkins-shared-lib') import com.company.pipeline.BuildUtils

node {
    def utils = new BuildUtils(this)
    stage('Build') {
        utils.buildDockerImage('myapp', '1.0.0')
    }
}
```

💬 Short Crisp Interview Answer

> *"Scripted Pipeline is the original Jenkins pipeline syntax, written in pure Groovy within `node {}` blocks. Unlike Declarative Pipeline's fixed structure, Scripted gives you full Groovy language capabilities — loops, conditionals, dynamic stage names, complex error handling with try-catch-finally, and node switching mid-pipeline. You use it when Declarative's `when {}` and `script {}` escape hatch isn't sufficient — for example, generating stage names dynamically from a list, complex retry logic with custom backoff, or building a pipeline that varies its structure based on runtime data. The tradeoff: no built-in structure validation, harder to read for non-Groovy developers, and IDE support is weaker. In practice, most new pipelines should use Declarative with `script {}` blocks for complex logic."*

⚠ Tricky Edge Cases & Gotchas

- **⚠ The closure variable capture bug.** In `services.each { service -> buildSteps[service] = { sh "build ${service}" } }` — by the time the closures run, `service` has changed to the last value in the list (loop variable captured by reference in Groovy). Fix: `def svc = service` inside the loop before using it in the closure.
- **`currentBuild.result` vs exceptions.** Setting `currentBuild.result = 'FAILURE'` does NOT stop execution. You must also `throw e` to actually abort the build. Many developers set the

result but forget to throw, causing the pipeline to continue past a failure.

- `node {}` **blocks create nested workspaces.** If you call `node {}` inside another `node {}`, Jenkins allocates a SECOND executor and workspace. Two executors are consumed, which can cause queue issues. Be intentional about when you switch node contexts.
- **No automatic `post {}` equivalent in Scripted.** The `finally {}` block is the closest equivalent, but it runs on every code path — including when the script itself has syntax errors before `try {}` is reached. Wrapping the entire pipeline in try-catch-finally is the pattern, but it requires discipline.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Declarative Pipeline (3.2)	Alternative syntax — know both and the decision criteria
Declarative vs Scripted (3.4)	Direct comparison of both approaches
Shared Libraries (3.13)	Libraries commonly used as helper functions in Scripted
Parallel Stages (3.7)	Scripted parallel has different syntax to Declarative

---

Topic 3.4 — Declarative vs Scripted ⚠

🟡 Intermediate | The Critical Comparison

📌 What It Is — In Simple Terms

This is one of the most commonly asked Jenkins questions. The ability to compare both approaches confidently, know their differences precisely, and make the right recommendation for a given context is what interviewers test.

⚙️ Side-By-Side Comparison

Same pipeline in both syntaxes:

```
// ═══════════════════════════════════════════════════════════
// DECLARATIVE PIPELINE
// ═══════════════════════════════════════════════════════════
pipeline {
    agent { label 'linux' }

    environment {
        APP_NAME = 'myapp'
        VERSION  = '1.0.0'
    }

    options {
```

```
        timeout(time: 30, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '10'))
    }

    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
            post {
                always {
                    junit 'target/surefire-reports/*.xml'
                }
            }
        }

        stage('Docker Build') {
            when { branch 'main' }
            steps {
                sh "docker build -t ${APP_NAME}:${VERSION} ."
                sh "docker push ${APP_NAME}:${VERSION}"
            }
        }
    }

    post {
        failure {
            slackSend channel: '#alerts', message: "Build failed:
${env.JOB_NAME}"
        }
        always {
            cleanWs()
        }
    }
}

// ═══════════════════════════════════════════════════════════════
// SCRIPTED PIPELINE — same logic, different structure
// ═══════════════════════════════════════════════════════════════
node('linux') {
    def APP_NAME = 'myapp'
    def VERSION  = '1.0.0'

    timeout(time: 30, unit: 'MINUTES') {  // Options are function calls in
Scripted

        try {
            stage('Build') {
                sh 'mvn clean package'
                // No built-in post{} — use try-finally per stage or
overall
                junit 'target/surefire-reports/*.xml'
            }
```

```
            stage('Docker Build') {
                if (env.BRANCH_NAME == 'main') {   // when{} becomes if()
                    sh "docker build -t ${APP_NAME}:${VERSION} ."
                    sh "docker push ${APP_NAME}:${VERSION}"
                }
            }

        } catch (Exception e) {
            slackSend channel: '#alerts',
                      message: "Build failed: ${env.JOB_NAME}"
            currentBuild.result = 'FAILURE'
            throw e
        } finally {
            cleanWs()
        }
    }
}
```

📊 Comprehensive Comparison Table

Feature	Declarative	Scripted
Syntax	Structured DSL in `pipeline {}`	Pure Groovy in `node {}`
Learning curve	Low — readable, documented	Higher — requires Groovy knowledge
Validation	Pre-execution syntax validation	No pre-validation — fails at runtime
IDE support	Good — intelliJ, VS Code plugins	Basic
Dynamic stage names	❌ Not supported	✅ Supported (variables in stage())
Stages inside loops	❌ Not natively	✅ stages in .each{} loops
`when` conditions	✅ Built-in `when {}` block	❌ Use Groovy `if()`
`post` actions	✅ Built-in `post {}` with conditions	❌ Manual try-catch-finally
`options` block	✅ Declarative `options {}`	❌ Function wrappers (timeout(), retry())
Full Groovy	❌ Via `script {}` escape hatch	✅ Everywhere
Error handling	✅ Automatic via `post { failure }`	Manual try-catch required
Parallel stages	✅ `parallel {}` block	✅ `parallel [:]` map
Recommended for	Most new pipelines	Complex logic, dynamic pipelines
Stage visualization	✅ Full	✅ Full

🔑 When to Choose Each

```
Use DECLARATIVE when:
  ✅ Standard CI/CD pipeline (checkout, build, test, package, deploy)
  ✅ Team has non-Groovy developers who need to read the pipeline
  ✅ You want pipeline syntax validation before execution
  ✅ Using `when` conditions, `input` gates, `options`, `post` heavily
  ✅ This covers 85%+ of real-world pipeline use cases

Use SCRIPTED when:
  ✅ Dynamic stage names required (stage names determined at runtime)
  ✅ Stages must be generated from data (list of services, environments)
  ✅ Complex retry logic beyond what retry() provides
  ✅ Multi-platform builds requiring node{} switching mid-pipeline
  ✅ Complex Groovy objects/data transformations throughout pipeline
  ✅ Using advanced Groovy patterns (closures, recursive methods)
  ✅ Calling complex Shared Library class methods throughout

Use DECLARATIVE + script{} when:
  ✅ 90% standard pipeline + 10% complex Groovy logic
  ✅ Best of both worlds — structured with Groovy escape hatch
  ✅ This is the most common real-world pattern
```

💬 Short Crisp Interview Answer

> *"Declarative and Scripted are two syntaxes for Jenkins Pipeline. Declarative uses a structured `pipeline {}` block with fixed sections — agent, environment, options, stages, post — and is validated before execution. It's the right choice for standard CI/CD pipelines and is easier to read. Scripted uses pure Groovy in `node {}` blocks with no structure enforcement — more powerful but requires Groovy knowledge and has no pre-execution validation. The key capability differences: Scripted supports dynamic stage names and stages inside loops (Declarative doesn't), while Declarative provides built-in `when {}` conditions, `post {}` handlers, and `options {}` without manual Groovy wrappers. In practice, I use Declarative for 90% of pipelines, with `script {}` blocks for complex Groovy logic where needed. Pure Scripted pipelines are reserved for genuinely complex cases — like a pipeline that generates its stage structure dynamically from a services manifest file."*

🔬 Deep Dive Answer

Mixing Declarative and Scripted:

```
// HYBRID APPROACH — Declarative structure + Scripted flexibility

pipeline {
    agent any
    stages {
        stage('Dynamic Build') {
            steps {
```

```
                script {
                    // Full Groovy inside Declarative step
                    def services =
readFile('services.txt').readLines().findAll { it.trim() }
                    def buildTasks = [:]

                    services.each { svc ->
                        def s = svc  // capture!
                        buildTasks[s] = {
                            sh "docker build -t ${s}:${BUILD_NUMBER}
services/${s}"
                        }
                    }

                    parallel buildTasks  // parallel inside script{}
inside Declarative
                }
            }
        }

        stage('Deploy') {
            when {
                branch 'main'  // Declarative when{} — clean and readable
            }
            steps {
                script {
                    // Complex logic in script{} — but structure from
Declarative
                    def manifest = readJSON file: 'deploy-manifest.json'
                    manifest.services.each { svc ->
                        retry(3) {
                            sh "helm upgrade --install ${svc.name}
./charts/${svc.chart} \
                                --set image.tag=${BUILD_NUMBER}"
                        }
                    }
                }
            }
        }
    }

    post {
        // Declarative post{} handles all cleanup — no try-catch needed
        always { cleanWs() }
        failure { slackSend channel: '#alerts', message: "Failed:
${JOB_NAME}" }
    }
}
```

The `@NonCPS` Annotation — When Groovy Meets CPS:

```
// Certain Groovy methods are NOT serializable by the CPS engine
// Using them in pipeline code causes: "java.io.NotSerializableException"
// Fix: mark the method @NonCPS — runs outside CPS engine (not resumable)

@NonCPS
def parseDeployManifest(String json) {
    // JsonSlurper, regex matchers, complex list operations
    // These are NOT CPS-safe — must be in @NonCPS method
    def slurper = new groovy.json.JsonSlurper()
    return slurper.parseText(json)
        .services
        .findAll { it.enabled }
        .collect { it.name }
}

pipeline {
    agent any
    stages {
        stage('Parse Config') {
            steps {
                script {
                    def configJson = readFile('deploy-config.json')
                    def services = parseDeployManifest(configJson)  //
calls @NonCPS method
                    // services is now a plain List — CPS-safe
                    echo "Services to deploy: ${services}"
                }
            }
        }
    }
}
```

❓ Common Interview Questions & Strong Answers

Q1: ⚠ A pipeline needs to read a list of microservices from a file and create a deployment stage for each one dynamically. Which syntax do you use and why?

> This requires Scripted Pipeline or Declarative with a `script {}` block. Declarative doesn't support dynamic stage names or stages generated in loops at the `stage()` level. My approach: use Declarative for overall structure (options, post, agent), and inside the relevant `steps {}`, use `script {}` with a Groovy loop that builds a parallel map and calls `parallel`. If the stage names themselves need to be dynamic (not just the work inside them), I use full Scripted Pipeline. Example: `def buildTasks = [:]; services.each { svc -> buildTasks[svc] = { sh "deploy ${svc}" } }; parallel buildTasks`. Note the critical Groovy gotcha: capture loop variable with `def s = svc` before using in the closure.

Q2: I'm building a new pipeline for a standard Java service — CI + deploy to staging + deploy to production. Which syntax?

> Declarative, without question. It's a standard use case with named stages, `when { branch 'main' }` for production gate, `input {}` for manual approval, `post { failure {} always {} }` for notifications. The structure is self-documenting and any developer can read it. I'd add a `script {}` block only if I need specific Groovy logic like parsing a version file or constructing a dynamic Helm values map. Scripted Pipeline would add complexity here with no benefit.

⚠ Tricky Edge Cases & Gotchas

- **You can't use `environment {}` block at the `node {}` level in Scripted.** Scripted uses `env.VAR_NAME = 'value'` or local `def` variables. Mixing Declarative `environment {}` syntax into Scripted code is a syntax error.
- **Declarative `when {}` evaluates before the stage agent allocates.** If the stage has its own `agent {}` and a `when {}` condition, the condition is checked before any agent is provisioned. This is correct behavior but surprises people expecting the stage environment to be available.
- **`post { always {} }` in Declarative runs even if the pipeline is aborted.** A developer aborting a long-running pipeline will still trigger `always {}` — so cleanup and certain notifications run. This is usually desirable but can cause confusion if notifications fire for manual aborts.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Declarative Pipeline (3.2)	Full Declarative reference
Scripted Pipeline (3.3)	Full Scripted reference
Shared Libraries (3.13)	Libraries work with both; class-based libraries favor Scripted
CPS Internals (2.3)	@NonCPS applies to both syntaxes

Topic 3.5 — Jenkinsfile: Anatomy, Best Practices, Version Control

🟢 Beginner | The Fundamental Artifact

📌 What It Is — In Simple Terms

The **Jenkinsfile** is the text file (typically named `Jenkinsfile`) that contains the Pipeline definition. It lives at the root of your repository, is written in Groovy DSL (Declarative or Scripted), and is the primary artifact of Pipeline as Code.

Every Jenkinsfile is a Jenkins pipeline. Every pipeline should be a Jenkinsfile.

⚙️ Jenkinsfile Anatomy — Full Production Example

```
//
═══════════════════════════════════════════════════════════════════════
// PRODUCTION JENKINSFILE — Annotated anatomy
// Application: myservice (Java Spring Boot + Docker + Kubernetes)
//
═══════════════════════════════════════════════════════════════════════

// 1. SHARED LIBRARY IMPORT (if using org-wide pipeline library)
@Library('jenkins-shared-library@v2.4.0') _   // Pin to version for
stability

// 2. PIPELINE BLOCK — must be the top-level structure for Declarative
pipeline {

    // 3. AGENT — defines where the WHOLE pipeline runs
    //    Use 'none' + per-stage agents when stages need different
environments
    agent none

    // 4. OPTIONS — pipeline-wide configuration
    options {
        buildDiscarder(logRotator(
            numToKeepStr:          '20',
            daysToKeepStr:         '30',
            artifactNumToKeepStr:  '5',
            artifactDaysToKeepStr: '7'
        ))
        timeout(time: 60, unit: 'MINUTES')
        timestamps()
        disableConcurrentBuilds(abortPrevious: true)  // Cancel older
branch builds
        durabilityHint('PERFORMANCE_OPTIMIZED')
    }

    // 5. ENVIRONMENT — pipeline-wide environment variables
    environment {
        APP_NAME    = 'myservice'
        REGISTRY    = 'registry.example.com/myorg'
        // Version: semantic version from file + git SHA
        APP_VERSION = sh(script: 'cat VERSION', returnStdout: true).trim()
        IMAGE_TAG   = "${APP_VERSION}-${env.GIT_COMMIT.take(7)}"
        FULL_IMAGE  = "${REGISTRY}/${APP_NAME}:${IMAGE_TAG}"
    }

    // 6. PARAMETERS — build-time inputs
    parameters {
        booleanParam(name: 'FORCE_DEPLOY',      defaultValue: false,
                     description: 'Force deploy even on non-main
branches')
        choice(name: 'LOG_LEVEL',              choices: ['INFO', 'DEBUG'],
               description: 'Application log level')
        string(name: 'HELM_EXTRA_ARGS',        defaultValue: '',
               description: 'Extra helm arguments (e.g., --dry-run)')
```

```
    }

    // 7. TRIGGERS - automated build conditions
    triggers {
        githubPush()   // Fire on GitHub webhook
    }

    // 8. STAGES - the pipeline logic
    stages {

        stage('CI') {
            agent {
                kubernetes {
                    // Reference pod template from shared library or
inline
                    inheritFrom 'maven-jdk17'
                }
            }
            stages {
                stage('Checkout & Validate') {
                    steps {
                        checkout scm
                        // Validate important files exist
                        sh 'test -f pom.xml || { echo "pom.xml missing";
exit 1; }'
                        sh 'test -f Dockerfile || { echo "Dockerfile
missing"; exit 1; }'
                        // Print build context info
                        sh '''
                            echo "=== Build Context ===="
                            echo "Branch: ${BRANCH_NAME}"
                            echo "Commit: ${GIT_COMMIT}"
                            echo "Version: ${APP_VERSION}"
                            echo "Image: ${FULL_IMAGE}"
                            echo "====================="
                        '''
                    }
                }

                stage('Test') {
                    steps {
                        container('maven') {
                            sh 'mvn test -B -
Dmaven.test.failure.ignore=false'
                        }
                    }
                    post {
                        always {
                            junit          allowEmptyResults: true,
                                           testResults: 'target/surefire-
reports/*.xml'
                            publishCoverage adapters: [
                                jacocoAdapter(path:
'target/site/jacoco/jacoco.xml',
```

```
                                                    thresholds:
[[thresholdTarget: 'LINE',

unhealthyThreshold: 70.0,

unstableThreshold: 80.0]])
                            ]
                        }
                    }
                }

                stage('Build & Push') {
                    steps {
                        container('kaniko') {
                            sh """
                                /kaniko/executor \
                                    --context=dir:///workspace \
                                    --dockerfile=/workspace/Dockerfile \
                                    --destination=${FULL_IMAGE} \
                                    --cache=true \
                                    --cache-
repo=${REGISTRY}/${APP_NAME}/cache
                            """
                        }
                    }
                }
            }
        }

        stage('Deploy: Staging') {
            when {
                anyOf {
                    branch 'main'
                    branch 'release/*'
                    expression { return params.FORCE_DEPLOY }
                }
            }
            agent { label 'helm-deploy' }
            environment {
                TARGET_NS = 'staging'
            }
            steps {
                withCredentials([file(credentialsId: 'kubeconfig-staging',
                                      variable: 'KUBECONFIG')]) {
                    sh """
                        helm upgrade --install ${APP_NAME} ./chart \
                            --namespace ${TARGET_NS} \
                            --set image.repository=${REGISTRY}/${APP_NAME}
\
                            --set image.tag=${IMAGE_TAG} \
                            --set logLevel=${params.LOG_LEVEL} \
                            -f chart/values-staging.yaml \
                            --wait --timeout=5m \
                            --atomic \
```

```
                            ${params.HELM_EXTRA_ARGS}
                    """
                }
            }
        }

        stage('Approve: Production') {
            when { branch 'main' }
            steps {
                input(
                    message: "Deploy ${FULL_IMAGE} to PRODUCTION?",
                    ok:      'Approve & Deploy',
                    submitter: 'release-managers'
                )
            }
        }

        stage('Deploy: Production') {
            when { branch 'main' }
            agent { label 'helm-deploy' }
            environment { TARGET_NS = 'production' }
            steps {
                withCredentials([file(credentialsId: 'kubeconfig-
production',
                                      variable: 'KUBECONFIG')]) {
                    sh """
                        helm upgrade --install ${APP_NAME} ./chart \
                            --namespace ${TARGET_NS} \
                            --set image.repository=${REGISTRY}/${APP_NAME}
\
                            --set image.tag=${IMAGE_TAG} \
                            -f chart/values-production.yaml \
                            --wait --timeout=10m \
                            --atomic \
                            --history-max=5
                    """
                }
            }
            post {
                success {
                    // Record deployment in audit system
                    sh """
                        curl -X POST
https://deployments.internal/api/record \
                            -H 'Content-Type: application/json' \
                            -d
'{"service":"${APP_NAME}","version":"${IMAGE_TAG}","env":"production"}'
                    """
                }
            }
        }
    }

    // 9. GLOBAL POST — runs after ALL stages
```

```
    post {
        always {
            cleanWs(cleanWhenAborted: true,
                    cleanWhenFailure: false,  // Keep workspace on failure
for debugging
                    cleanWhenSuccess: true)
        }
        success {
            slackSend channel: '#deployments',
                      color:   'good',
                      message: "✅ *${APP_NAME}* `${IMAGE_TAG}`
deployed\nBranch: `${BRANCH_NAME}` | <${BUILD_URL}|Build
#${BUILD_NUMBER}>"
        }
        failure {
            slackSend channel: '#build-failures',
                      color:   'danger',
                      message: "❌ *${APP_NAME}* build FAILED\nBranch:
`${BRANCH_NAME}` | <${BUILD_URL}|Build #${BUILD_NUMBER}>"
        }
    }
}
```

🔑 Jenkinsfile Best Practices

```
1. LOCATION: Always at repository root
   → Jenkins Multi-Branch looks for Jenkinsfile at root by default
   → Custom path requires configuration: "Script Path" in job config
   → Exception: monorepo — Jenkinsfiles at service root level

2. VERSION CONTROL HYGIENE:
   → Never commit secrets — use Jenkins credentials() binding
   → Pin shared library versions: @Library('lib@v2.4.0') not @main
   → Add Jenkinsfile to protected files (can't delete without PR)

3. IDEMPOTENCY:
   → Running the same Jenkinsfile N times should produce the same result
   → Use --atomic in helm upgrade (rollback on failure)
   → Use --if-exists in kubectl commands

4. FAIL FAST:
   → Put fastest, cheapest checks first (lint before compile before test)
   → Set timeout on the entire pipeline AND on individual slow stages

5. READABILITY:
   → Name stages clearly: 'Test' not 'Stage 3'
   → Add echo statements at key decision points
   → Use environment{} for magic strings, not inline literals

6. CREDENTIALS:
```

```
   → NEVER: sh "curl -H 'Authorization: Bearer ${TOKEN}'" where TOKEN is
in env
   → ALWAYS: withCredentials([string(credentialsId: 'my-token', variable:
'TOKEN')]) { }

7. AGENT EFFICIENCY:
   → Use agent none + per-stage agents when stages need different tools
   → Use Kubernetes pod templates via Shared Libraries, not inline YAML
   → Clean workspace with cleanWs() post{always}

8. DRY:
   → Use Shared Libraries for repeated patterns across pipelines
   → Don't copy-paste Jenkinsfile sections — extract to library functions
```

💬 Short Crisp Interview Answer

> *"A Jenkinsfile is the pipeline definition file committed to source control, typically at the repository root. It's the core artifact of Pipeline as Code. Best practices: always version-control it with the application code; never hardcode secrets — use Jenkins credential binding; pin Shared Library versions to prevent unexpected behavior changes; set timeouts on both the whole pipeline and on individual slow stages; put fastest checks first; use `cleanWs()` in `post { always {} }` to prevent workspace accumulation. The Jenkinsfile should be readable enough that a developer who didn't write it can understand what the pipeline does in 30 seconds. If a Jenkinsfile requires a Groovy expert to understand it, it's too complex and should be refactored into Shared Library functions with descriptive names."*

⚠ Tricky Edge Cases & Gotchas

- **`Jenkinsfile` must be capitalized.** The default name is `Jenkinsfile` — case-sensitive on Linux filesystems. `jenkinsfile` or `JenkinsFile` will not be found by default (unless the Script Path is configured to match).
- **First-run `parameters {}` bootstrap.** On the very first run after adding a `parameters {}` block, the build runs without parameters (Jenkins hasn't read the new Jenkinsfile yet). The parameters appear in subsequent runs. For this reason, always provide safe `defaultValue` for every parameter.
- **Multi-branch pipeline `Jenkinsfile` for all branches.** Each branch uses its OWN `Jenkinsfile`. This means the `main` branch and `feature/new-checkout` can have different pipeline logic. This is powerful — feature branches can have lighter pipelines — but requires governance to ensure security-critical stages aren't removed from feature branches.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Declarative Pipeline (3.2)	Jenkinsfile IS the Declarative Pipeline
Pipeline as Code (1.6)	Jenkinsfile is the implementation of Pipeline as Code

Connected Topic	How It Connects
Multi-Branch Pipelines (4.1)	Multi-Branch auto-discovers Jenkinsfiles per branch
Shared Libraries (3.13)	Library functions called from Jenkinsfile
Credentials (3.12)	Secrets referenced from Jenkinsfile

Topic 3.6 — Stages, Steps, and Post Actions

🟢 Beginner | Pipeline Building Blocks

📌 What It Is — In Simple Terms

The three core structural elements of a Jenkins pipeline:

- **Stage** — a named logical group of work with a clear purpose (Build, Test, Deploy). Stages appear in the pipeline visualization. A pipeline without stages is just a list of commands.
- **Step** — a single action within a stage (run a command, call a plugin function, read a file). Steps are the atomic units of work.
- **Post** — cleanup, notification, and result-handling actions that run AFTER stages complete, conditional on the build result.

```
pipeline {
    stages {                    ← Contains all stages
        stage('Build') {        ← Named unit of work
            steps {             ← Container for steps
                sh 'mvn build'  ← A step: execute shell command
            }
        }
    }
    post {                      ← Post-build actions
        failure {
            slackSend "Failed"  ← Step in a post condition
        }
    }
}
```

⚙️ Steps — The Complete Reference

```
// ── SHELL COMMANDS ──────────────────────────────────────────────
sh 'mvn clean package'             // Execute shell command (Linux/Mac)
bat 'msbuild MyProject.sln'        // Windows batch command
powershell 'Get-Process jenkins'   // PowerShell command
```

```
// Capture shell output:
def version = sh(script: 'cat VERSION', returnStdout: true).trim()

// Check exit code without failing (returnStatus):
def exitCode = sh(script: 'command-that-might-fail', returnStatus: true)
if (exitCode != 0) { echo "Command failed with code ${exitCode}" }

// Multi-line shell (heredoc):
sh '''
    echo "Starting build..."
    mvn clean package
    echo "Build complete: $(date)"
'''

// Shell with environment variables:
sh """
    export IMAGE_TAG=${env.BUILD_NUMBER}
    docker build -t myapp:\$IMAGE_TAG .   # \$ escapes GString
interpolation
    docker push myapp:\$IMAGE_TAG
"""

// ── FILE OPERATIONS ──────────────────────────────────────────────
writeFile file: 'build-info.txt', text: "Version: ${env.BUILD_NUMBER}"
def content = readFile 'build-info.txt'
def exists = fileExists 'pom.xml'

// Read/write JSON:
def config = readJSON file: 'deploy-config.json'
config.version = env.BUILD_NUMBER
writeJSON file: 'deploy-config.json', json: config, pretty: 4

// Read/write YAML:
def helmValues = readYaml file: 'values.yaml'
writeYaml file: 'values-override.yaml', data: [image: [tag:
env.BUILD_NUMBER]]

// ── SCM OPERATIONS ───────────────────────────────────────────────
checkout scm   // Check out the SCM configured for this job

// Explicit git checkout:
git branch: 'main',
    url: 'https://github.com/myorg/myapp.git',
    credentialsId: 'github-creds'

// ── ARTIFACT OPERATIONS ──────────────────────────────────────────
// Archive artifacts to Jenkins build record:
archiveArtifacts artifacts: 'target/*.jar,target/*.war',
                 fingerprint: true,
                 onlyIfSuccessful: true,
                 allowEmptyArchive: false

// Publish test results (JUnit XML format):
junit testResults: 'target/surefire-reports/*.xml',
```

```
        allowEmptyResults: true,
        skipPublishingChecks: false

  // Publish HTML report:
  publishHTML([
      allowMissing: false,
      alwaysLinkToLastBuild: true,
      keepAll: true,
      reportDir: 'target/site',
      reportFiles: 'index.html',
      reportName: 'Test Coverage Report'
  ])

  // ── BUILD / PIPELINE CONTROL ──────────────────────────────────────
  // Mark build as unstable (test failures, warning thresholds):
  unstable(message: 'Test coverage below threshold')
  // equivalent: currentBuild.result = 'UNSTABLE'

  // Mark build as failed programmatically:
  error('Deployment health check failed — rolling back')
  // equivalent: throw new Exception('...')

  // Force build success regardless of earlier failures:
  // (use sparingly — defeats the point of CI)
  currentBuild.result = 'SUCCESS'

  // Add a description to build (visible in build list):
  currentBuild.description = "Version: ${env.APP_VERSION} →
  ${env.TARGET_ENV}"
  currentBuild.displayName = "#${env.BUILD_NUMBER} (v${env.APP_VERSION})"

  // Wait for a URL to become available:
  waitForUrl url: 'http://localhost:8080/health',
             timeout: 300,    // 5 minutes
             recurrence: 10   // check every 10 seconds

  // ── SLEEP AND TIMING ──────────────────────────────────────────────
  sleep time: 30, unit: 'SECONDS'
  sleep 30   // Shorthand — seconds

  // ── INPUT (manual approval gate) ──────────────────────────────────
  def approval = input(
      message:   'Approve deployment to production?',
      ok:        'Deploy',
      submitter: 'release-managers,sre-team',
      submitterParameter: 'APPROVER',   // Captures who approved
      parameters: [
          choice(name: 'STRATEGY', choices: ['canary', 'rolling'],
  description: 'Strategy')
      ]
  )
  echo "Approved by: ${approval['APPROVER']}, strategy:
  ${approval['STRATEGY']}"
```

```
// ── STASH / UNSTASH (cross-stage file transfer) ──────────────────
stash name: 'compiled-jar', includes: 'target/*.jar'
unstash 'compiled-jar'

// ── TRIGGER OTHER JOBS ───────────────────────────────────────────
// Trigger downstream job and wait for result:
build job: 'integration-test-pipeline',
      wait: true,
      parameters: [
          string(name: 'IMAGE_TAG', value: env.IMAGE_TAG),
          booleanParam(name: 'FULL_TEST', value: true)
      ]

// Trigger without waiting:
build job: 'notification-job', wait: false

// ── CONTAINER STEP (Kubernetes agents) ──────────────────────────
container('maven') {
    sh 'mvn clean package'
}
container('docker') {
    sh 'docker build ...'
}

// ── WITHCREDENTIALS (secure credential injection) ───────────────
withCredentials([
    usernamePassword(credentialsId: 'docker-creds',
                     usernameVariable: 'DOCKER_USER',
                     passwordVariable: 'DOCKER_PASS'),
    string(credentialsId: 'sonar-token', variable: 'SONAR_TOKEN'),
    file(credentialsId: 'kubeconfig', variable: 'KUBECONFIG'),
    sshUserPrivateKey(credentialsId: 'ssh-deploy-key',
                      keyFileVariable: 'SSH_KEY',
                      usernameVariable: 'SSH_USER')
]) {
    sh "echo ${DOCKER_PASS} | docker login -u ${DOCKER_USER} --password-
stdin"
    sh "ssh -i ${SSH_KEY} ${SSH_USER}@server.example.com 'deploy.sh'"
}

// ── TIMEOUT AND RETRY ────────────────────────────────────────────
timeout(time: 5, unit: 'MINUTES') {
    sh './slow-integration-test.sh'
}

retry(3) {
    sh 'kubectl rollout status deployment/myapp --timeout=60s'
}

// ── LOCK (prevent concurrent execution on a resource) ────────────
lock(resource: 'production-database', inversePrecedence: true) {
    sh 'liquibase update'  // Only one migration runs at a time
}
```

```
// ── WRAP (generic wrapper step) ──────────────────────────────────────
wrap([$class: 'AnsiColorBuildWrapper', 'colorMapName': 'xterm']) {
    sh 'colorized-test-runner.sh'
}
```

⚙️ Post Actions — Complete Reference

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps { sh 'mvn package' }

            // Stage-level post: runs after THIS STAGE specifically
            post {
                always {
                    // Always: publish test results even if build failed
                    junit 'target/surefire-reports/*.xml'
                }
                failure {
                    // Archive debugging artifacts on failure
                    archiveArtifacts artifacts: 'target/logs/**',
allowEmptyArchive: true
                }
            }
        }
    }

    // Pipeline-level post: runs after ALL stages complete
    post {
        // Run regardless of outcome (cleanup, always publish results)
        always {
            cleanWs()
            script {
                echo "Total build time: ${currentBuild.durationString}"
            }
        }

        // Build succeeded (green)
        success {
            slackSend channel: '#deployments', color: 'good',
                      message: "✅ Build passed: ${env.JOB_NAME}
#${env.BUILD_NUMBER}"
        }

        // Build failed (red)
        failure {
            slackSend channel: '#alerts', color: 'danger',
                      message: "❌ Build FAILED: ${env.JOB_NAME}
#${env.BUILD_NUMBER}"
```

```
            emailext subject: "Build Failed: ${env.JOB_NAME}",
                     body: "See ${env.BUILD_URL}",
                     to: "\${DEFAULT_RECIPIENTS}"
        }

        // Build unstable (yellow — tests failed but build didn't crash)
        unstable {
            slackSend channel: '#alerts', color: 'warning',
                      message: "⚠️ Build UNSTABLE: ${env.JOB_NAME} (test
failures)"
        }

        // Build was aborted manually
        aborted {
            echo "Pipeline manually aborted — no notification sent"
        }

        // Result CHANGED from last build (either direction)
        changed {
            slackSend channel: '#deployments',
                      message: "🔄 Build result changed:
${currentBuild.currentResult}"
        }

        // Changed from failure to success (build was FIXED)
        fixed {
            slackSend channel: '#deployments', color: 'good',
                      message: "🎉 Build FIXED: ${env.JOB_NAME} — back to
green"
        }

        // Changed from success to failure (build REGRESSED)
        regression {
            slackSend channel: '#alerts', color: 'danger',
                      message: "📉 Build REGRESSED: ${env.JOB_NAME} — was
passing"
            pagerduty(resolve: false, serviceKey: env.PAGERDUTY_KEY,
                      incDescription: "Build broken: ${env.JOB_NAME}")
        }

        // Build not built (skipped due to when conditions, etc.)
        notBuilt { echo "Build was skipped" }

        // Runs LAST after all other post conditions
        cleanup {
            echo "Final cleanup after pipeline execution"
            deleteDir() // Clean workspace after everything else
        }
    }
}
```

💬 Short Crisp Interview Answer

> *"Stages are named logical groups that appear in the pipeline visualization — they give structure and names to build phases like Build, Test, Deploy. Steps are the atomic actions within stages — shell commands, file operations, plugin functions, credential binding. Post is the cleanup and notification layer that runs after stages, conditional on build result. Post has multiple conditions: `always` for cleanup that runs regardless, `success` and `failure` for result-specific notifications, `changed` for "build result changed from last run," `fixed` for "build just turned green," and `regression` for "build just turned red." Stage-level post is the right place for publishing test results — you want JUnit results published even if the test stage failed, and at the stage level they're scoped correctly to that stage's output."*

⚠ Tricky Edge Cases & Gotchas

- **`post { failure {} }` at stage level vs pipeline level.** Stage-level failure post runs if THAT STAGE fails. Pipeline-level failure post runs if THE WHOLE PIPELINE fails (which includes any stage failing). Putting `junit` in stage-level `post { always {} }` is correct — test results are scoped to that stage.
- **`error()` in `post { }` blocks.** Calling `error()` inside a `post {}` block causes surprising behavior. If you call `error()` in `post { always {} }`, it overrides the original build result. The build that was SUCCESS now becomes FAILURE. Be careful with exceptions in post blocks.
- **`cleanWs(cleanWhenFailure: false)` is a debugging lifesaver.** By default `cleanWs()` cleans on all conditions. Setting `cleanWhenFailure: false` preserves the workspace when the build fails — letting you SSH to the agent and investigate. In production, this is a trade-off: debugging ability vs disk space.
- **Post blocks have access to build result via `currentBuild.currentResult`.** Inside `post { always {} }`, you can check `currentBuild.currentResult` to take different actions: `if (currentBuild.currentResult == 'FAILURE') { ... }`.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Declarative Pipeline (3.2)	stages/steps/post are Declarative building blocks
Parallel Stages (3.7)	`parallel {}` is a special step inside stages
Environment Variables (3.11)	`env.*` used inside steps
Credentials (3.12)	`withCredentials {}` is a step wrapper

Topic 3.7 — Parallel Stages ⚠

🟡 Intermediate | Speed and Efficiency

📌 What It Is — In Simple Terms

Parallel stages allow multiple stages to run simultaneously on different agents or threads, reducing total pipeline execution time. Instead of running lint → unit tests → security scan sequentially (3 minutes each = 9 minutes total), run all three in parallel (3 minutes total — the longest one determines total time).

This is one of the most heavily tested pipeline topics because it directly impacts build speed, which directly impacts developer productivity.

🔍 Why It Exists — The Problem It Solves

```
SEQUENTIAL (slow):
  Lint        → 2 min
  Unit Tests  → 5 min
  SAST Scan   → 3 min
  Dep Check   → 4 min
  ────────────────────
  Total:       14 min    ← 14 minutes before feedback

PARALLEL (fast):
  Lint  ]
  Tests ] → all simultaneously → 5 min (longest wins)
  SAST  ]
  Deps  ]
  ────────────────────
  Total:  5 min           ← 5 minutes — same confidence, 3x faster
```

⚙️ Parallel Stages — Complete Syntax Reference

Declarative Parallel Syntax

```
pipeline {
    agent none   // No global agent — each parallel stage gets its own

    stages {

        // ── BASIC PARALLEL ──────────────────────────────────────────
        stage('Quality Checks') {
            parallel {
                stage('Unit Tests') {
                    agent { label 'java' }
                    steps {
                        sh 'mvn test -Psuite-unit'
                    }
                    post {
                        always { junit 'target/surefire-
reports/unit/*.xml' }
```

```
                    }
                }

                stage('SAST Scan') {
                    agent { label 'linux' }
                    steps {
                        sh 'semgrep --config auto src/ --json > sast-
results.json'
                    }
                    post {
                        always {
                            archiveArtifacts artifacts: 'sast-
results.json', allowEmptyArchive: true
                        }
                    }
                }

                stage('Dependency Check') {
                    agent { label 'linux' }
                    steps {
                        sh 'mvn dependency-check:check'
                    }
                    post {
                        always {
                            publishHTML([reportDir: 'target', reportFiles:
'dependency-check-report.html',
                                         reportName: 'Dependency
Vulnerability Report'])
                        }
                    }
                }

                stage('Secret Detection') {
                    agent { label 'linux' }
                    steps {
                        sh 'gitleaks detect --source . --report-format
json --report-path gitleaks-report.json'
                    }
                }
            }
        }

        // ── PARALLEL WITH FAIL-FAST ──────────────────────────────────
        // failFast: if ANY parallel branch fails, CANCEL all others
immediately
        stage('Multi-Platform Build') {
            failFast true    // ← CRITICAL option — stop all others on
first failure
            parallel {
                stage('Build: Linux') {
                    agent { label 'linux' }
                    steps { sh 'make build-linux' }
                }
                stage('Build: Windows') {
```

```
                    agent { label 'windows' }
                    steps { bat 'msbuild /p:Configuration=Release' }
                }
                stage('Build: MacOS') {
                    agent { label 'macos' }
                    steps { sh 'xcodebuild -scheme MyApp' }
                }
            }
        }
    }
}
```

Parallel with Same Agent (Kubernetes Multi-Container)

```
pipeline {
    agent {
        kubernetes {
            yaml '''
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: jnlp
    image: jenkins/inbound-agent:latest-jdk17
  - name: maven
    image: maven:3.9-eclipse-temurin-17
    command: ["cat"]
    tty: true
  - name: node
    image: node:18-alpine
    command: ["cat"]
    tty: true
  - name: trivy
    image: aquasec/trivy:latest
    command: ["cat"]
    tty: true
'''
        }
    }

    stages {
        stage('Parallel Scans') {
            parallel {
                stage('Java Tests') {
                    steps {
                        container('maven') {
                            sh 'mvn test'
                        }
                    }
                }
                stage('Frontend Tests') {
```

```
                    steps {
                        container('node') {
                            sh 'npm test -- --watchAll=false'
                        }
                    }
                }
                stage('Container Scan') {
                    steps {
                        container('trivy') {
                            sh 'trivy image --exit-code 1 --severity
HIGH,CRITICAL myapp:latest'
                        }
                    }
                }
            }
        }
    }
}
```

Dynamic Parallel (Scripted inside Declarative)

```
pipeline {
    agent any
    stages {
        stage('Test All Services') {
            steps {
                script {
                    // Build parallel map dynamically
                    def services = ['auth', 'payment', 'order',
'notification', 'inventory']
                    def parallelStages = [:]

                    // CRITICAL: capture loop variable before closure
                    services.each { service ->
                        def svc = service  // ← MUST capture — Groovy
closure variable capture!
                        parallelStages["Test: ${svc}"] = {
                            node('linux') {
                                stage("Test: ${svc}") {
                                    checkout scm
                                    sh "mvn test -pl services/${svc}"
                                    junit
"services/${svc}/target/surefire-reports/*.xml"
                                }
                            }
                        }
                    }

                    // failFast in script block:
                    parallelStages.failFast = true
                    parallel parallelStages
```

```
                }
            }
        }

        // ── DYNAMIC PARALLEL FROM FILE ──────────────────────────────
        stage('Deploy to Test Environments') {
            steps {
                script {
                    // Read environment list from config file
                    def envConfig = readJSON file: 'environments.json'
                    def deployStages = [:]

                    envConfig.environments.each { env ->
                        def e = env  // capture!
                        deployStages["Deploy: ${e.name}"] = {
                            stage("Deploy: ${e.name}") {
                                sh """
                                    helm upgrade --install myapp ./chart \
                                        --namespace ${e.namespace} \
                                        --set image.tag=${BUILD_NUMBER} \
                                        -f ${e.valuesFile}
                                """
                            }
                        }
                    }

                    parallel deployStages
                }
            }
        }
    }
}
```

Parallel Test Sharding

```
// Split a large test suite across N parallel agents
// Each agent runs 1/N of the tests

pipeline {
    agent none
    stages {
        stage('Test (Sharded)') {
            parallel {
                stage('Shard 1/4') {
                    agent { label 'java' }
                    steps {
                        checkout scm
                        sh 'mvn test -
Dsurefire.includes="A*Test.java,B*Test.java,C*Test.java"'
                    }
                    post { always { junit 'target/surefire-reports/*.xml'
```

```
} }
                }
                stage('Shard 2/4') {
                    agent { label 'java' }
                    steps {
                        checkout scm
                        sh 'mvn test -
Dsurefire.includes="D*Test.java,E*Test.java,F*Test.java"'
                    }
                    post { always { junit 'target/surefire-reports/*.xml'
} }
                }
                stage('Shard 3/4') {
                    agent { label 'java' }
                    steps {
                        checkout scm
                        sh 'mvn test -
Dsurefire.includes="G*Test.java,H*Test.java,I*Test.java"'
                    }
                    post { always { junit 'target/surefire-reports/*.xml'
} }
                }
                stage('Shard 4/4') {
                    agent { label 'java' }
                    steps {
                        checkout scm
                        sh 'mvn test -
Dsurefire.includes="J*Test.java,K*Test.java"'
                    }
                    post { always { junit 'target/surefire-reports/*.xml'
} }
                }
            }
        }
    }
}
```

🔑 Key Concepts

Concept	Meaning
Parallel stages	Multiple stages running concurrently — each needs a free executor
`failFast true`	Cancel all running parallel branches if any one fails
Parallel branch	One concurrent execution path in a `parallel {}` block
Dynamic parallel	Parallel branches built programmatically from a list/map in `script {}`
Executor cost	Each parallel branch consumes one executor — ensure enough agents

Concept	Meaning
Closure variable capture	The Groovy loop variable bug — must do `def s = service` before closures
Stage naming in parallel	Parallel stage names appear in Blue Ocean visualization
Longest branch	Total parallel time = longest individual branch time

---

💬 Short Crisp Interview Answer

> *"Parallel stages in Jenkins allow multiple stages to run concurrently, reducing pipeline time from the sum of all stages to the time of the longest stage. In Declarative Pipeline, the `parallel {}` block inside a `stage {}` defines parallel sub-stages, each getting their own executor and optional agent. The `failFast true` option cancels all running branches immediately when any one fails — critical for expensive parallel stages like multi-platform builds where there's no point building MacOS if Linux already failed. For dynamic parallelism — building a parallel map from a list of services — you use a `script {}` block with a Groovy map where keys are stage names. The critical gotcha: loop variable capture in Groovy closures. Always do `def s = service` inside the loop before using `service` in the closure body, or all branches will use the last value of the loop variable."*

🔬 Deep Dive Answer

`failFast` Behavior in Detail:

```
stage('Multi-Platform') {
    failFast true
    parallel {
        stage('Linux') {
            steps {
                sleep 5
                sh 'exit 0'  // succeeds after 5s
            }
        }
        stage('Windows') {
            steps {
                sleep 2
                sh 'exit 1'  // FAILS after 2s
            }
        }
        stage('MacOS') {
            steps {
                sleep 10  // Would take 10s — but cancelled when Windows
fails at 2s
                sh 'xcodebuild'
            }
        }
    }
}
```

```
// With failFast true:
//   2s: Windows fails
//   2s: MacOS is ABORTED (cancelled) immediately
//   2s: Linux continues running (already in progress, can't be stopped
mid-step)
//        → when Linux step completes, stage reports as FAILED
//
// Without failFast:
//   All three run to completion regardless of individual failures
//   5s: Linux done (success)
//   2s: Windows done (failure — but others keep running)
//   10s: MacOS done
//   Total: 10s, result: FAILED

// IMPORTANT: failFast does NOT kill a running step mid-execution
// It only prevents NEW steps from starting in the cancelled branches
// A 30-minute test that's already running won't be killed
```

Parallel Stage Resource Planning:

```
Each parallel branch needs ONE executor (one agent or agent executor slot)

Example: 5 parallel stages
  Agent pool: 3 agents × 2 executors = 6 total executor slots
  5 parallel stages need 5 slots — fits ✅

  Agent pool: 2 agents × 2 executors = 4 total executor slots
  5 parallel stages need 5 slots
  → Jenkins runs 4 parallel, queues the 5th
  → 5th runs after first of the 4 finishes
  → Total time increases ❌

Rule: parallel stage count ≤ available executor slots for maximum
throughput
For Kubernetes agents: each parallel branch gets its own pod → scales
naturally
```

Parallel Post-Processing — Collecting Results:

```
stage('Tests') {
    steps {
        script {
            def testResults = [:]
            def parallelTests = [:]

            ['unit', 'integration', 'contract'].each { suite ->
                def s = suite
                parallelTests[s] = {
                    try {
```

```
                        sh "mvn test -P${s}"
                        testResults[s] = 'PASSED'
                    } catch (e) {
                        testResults[s] = 'FAILED'
                        currentBuild.result = 'FAILURE'
                        // Don't re-throw - collect all results first
                    }
                }
            }

            parallel parallelTests

            // After all parallel branches complete:
            echo "Test Results Summary:"
            testResults.each { suite, result ->
                echo "  ${suite}: ${result}"
            }

            // Now fail the build if any suite failed:
            if (testResults.values().contains('FAILED')) {
                error("One or more test suites failed - see summary
above")
            }
        }
    }
}
```

🏭 Real-World Production Example

Spotify's Parallel Pipeline Optimization: Spotify had a CI pipeline taking 22 minutes sequentially: lint (3min) + unit tests (8min) + integration tests (6min) + security scan (4min) + docker build (5min). After parallelizing: lint, unit tests, and security scan run together (8 min) → docker build (5 min) → integration tests (6 min). Total: 19 minutes → 8 minutes (with sequential docker build prerequisite). The key insight: security scan and lint don't depend on the build — they run on source code and can start immediately.

❓ Common Interview Questions & Strong Answers

Q1: ⚠ What is the closure variable capture bug in parallel stages and how do you fix it?

> In Groovy, closures capture variables by REFERENCE, not by value. In a loop like `services.each { service -> parallelTasks[service] = { sh "build ${service}" } }`, by the time the closures execute, the loop has finished and `service` points to the last value in the list. All closures execute with the same last value. Fix: add `def s = service` INSIDE the loop body before the closure: `services.each { service -> def s = service; parallelTasks[s] = { sh "build ${s}" } }`. Now each closure captures its own local `s` variable that doesn't change. This is one of the most common bugs in dynamic parallel Jenkins pipelines.

Q2: When should you use `failFast true` and when should you NOT use it?

> Use `failFast true` when: the parallel stages are expensive (long-running builds, cloud resource costs) and there's no value in completing them if one already failed — for example, multi-platform builds where all platforms must pass. If Linux fails, there's no point burning 20 more minutes on Windows and MacOS. Don't use `failFast` when: you want to collect ALL results before deciding — like running multiple test suites in parallel and wanting a full report of which suites passed and which failed. With `failFast true`, you'd only know which suite failed first, not all failures. For test suites, I usually disable `failFast` and manually collect results.

Q3: How many parallel stages can you run simultaneously?

> It's bounded by available executor slots. Each parallel branch requires one executor on one agent. If you have 10 parallel branches but only 4 executor slots available, Jenkins runs 4 in parallel and queues the rest, running them as slots free up — so total time is not simply "the longest branch." For Kubernetes agents, each branch gets its own pod — scaling is limited by cluster capacity, not pre-provisioned agents. Best practice: design parallel stage counts to match typical available capacity, or use Kubernetes agents for elastic scaling.

⚠ Tricky Edge Cases & Gotchas

- **⚠ The Groovy closure variable capture bug** — see Q1 above. This is the #1 parallel stages bug in production.
- **`failFast` in Declarative parallel is set in the parent `stage`, NOT inside `parallel`.** The syntax is: the `failFast true` directive goes DIRECTLY inside the `stage {}` that contains the `parallel {}` block, as a sibling to `parallel`, not inside `parallel`. Getting this placement wrong is a common syntax error.
- **Parallel stages on the same agent don't truly run in parallel.** If you use `agent any` and all parallel branches pick the same agent with only 1 free executor, they queue and run sequentially despite the `parallel {}` block. Always use `agent none` + per-stage agents for true parallelism.
- **Shared workspace in parallel.** If parallel branches run on the same agent (same workspace), they can overwrite each other's files. Always use separate workspaces (separate agents, or clearly separated directories within the workspace).
- **Post actions in parallel branches run in parallel too.** If 3 parallel branches all have `post { always { junit ... } }`, all three JUnit publish steps run in parallel. This is usually fine but can cause race conditions with shared resources (like writing to the same artifact storage path).

🔗 Connections to Other Topics

Connected Topic	How It Connects
Agent Types (2.4)	Parallel stages require multiple agent executors
Stages/Steps (3.6)	Parallel is a special form of stages
Stash/Unstash (3.10)	Parallel branches may need stash to share artifacts
Jenkins at Scale (2.8)	Parallel stages multiply agent demand — scaling concern
Build Time Optimization (8.2)	Parallelism is primary optimization technique

---

Topic 3.8 — Pipeline Parameters

🟡 Intermediate | User Inputs at Build Time

📌 What It Is — In Simple Terms

Pipeline Parameters are inputs you define in a Jenkinsfile that users (or automated systems) can provide when triggering a build. They turn a static pipeline into a flexible, reusable one — the same Jenkinsfile can deploy to `dev`, `staging`, or `production` based on a parameter, rather than requiring three separate pipelines.

Parameters are defined once in the `parameters {}` block and accessed anywhere in the pipeline as `params.PARAMETER_NAME`.

🔍 Why Parameters Exist — The Problems They Solve

Problem Without Parameters	Solution With Parameters
Need 3 separate pipelines for 3 environments	One pipeline, `TARGET_ENV` parameter
Can't skip tests for emergency hotfixes	`SKIP_TESTS` boolean parameter
Hard-coded version numbers	`APP_VERSION` string parameter
Can't target specific tests	`TEST_SUITE` choice parameter
No way to pass deployment approval tokens	`password` parameter type

⚙️ All Parameter Types — Complete Reference

```
pipeline {
    agent any

    parameters {

        // ── STRING: free-form text input ──────────────────────────────
        string(
            name:         'IMAGE_TAG',
            defaultValue: 'latest',
            description:  'Docker image tag to deploy (e.g., 1.2.3-
abc1234)'
        )

        // ── TEXT: multi-line string ───────────────────────────────────
        text(
            name:         'RELEASE_NOTES',
            defaultValue: '',
            description:  'Release notes for this deployment (multi-line
```

```
supported)'
        )

        // ── BOOLEAN: true/false checkbox ──────────────────────────────
        booleanParam(
            name:         'SKIP_TESTS',
            defaultValue: false,
            description:  'Skip test execution (use only for emergency
hotfixes)'
        )

        booleanParam(
            name:         'DRY_RUN',
            defaultValue: true,       // Default to safe mode
            description:  'Dry run — show what would be deployed without
deploying'
        )

        // ── CHOICE: dropdown of predefined options ────────────────────
        choice(
            name:    'TARGET_ENV',
            choices: ['dev', 'staging', 'production'],   // First is
default
            description: 'Target deployment environment'
        )

        choice(
            name:    'LOG_LEVEL',
            choices: ['INFO', 'DEBUG', 'WARN', 'ERROR'],
            description: 'Application log level'
        )

        // ── PASSWORD: masked input ────────────────────────────────────
        password(
            name:         'DEPLOY_APPROVAL_CODE',
            defaultValue: '',
            description:  'Authorization code for production deployment'
        )

        // ── FILE: upload a file as parameter (requires pipeline-input-
step plugin) ──
        file(
            name:        'CONFIG_FILE',
            description: 'Optional override configuration file'
        )

        // ── RUN: reference to another job's specific build ────────────
        run(
            name:    'UPSTREAM_BUILD',
            project: 'myapp-build-pipeline',
            description: 'Build to promote to production'
        )
    }
```

```
    // Access parameters anywhere using params.NAME
    environment {
        // Compute derived values from parameters
        IS_PRODUCTION = "${params.TARGET_ENV == 'production'}"
        HELM_DRY_RUN  = "${params.DRY_RUN ? '--dry-run' : ''}"
    }

    stages {
        stage('Validate Parameters') {
            steps {
                script {
                    // Parameter validation — fail fast on bad inputs
                    if (params.TARGET_ENV == 'production' &&
params.IMAGE_TAG == 'latest') {
                        error("❌ Cannot deploy 'latest' tag to production
— use a pinned version")
                    }

                    if (params.TARGET_ENV == 'production' &&
params.SKIP_TESTS) {
                        error("❌ Cannot skip tests for production
deployment")
                    }

                    if (params.IMAGE_TAG ==~ /^[0-9a-f]{7,40}$/) {
                        echo "✅ Deploying commit SHA:
${params.IMAGE_TAG}"
                    } else if (params.IMAGE_TAG ==~ /^\d+\.\d+\.\d+.*/) {
                        echo "✅ Deploying semantic version:
${params.IMAGE_TAG}"
                    } else if (params.IMAGE_TAG != 'latest') {
                        error("❌ IMAGE_TAG must be a git SHA, semver, or
'latest'")
                    }
                }
            }
        }

        stage('Build') {
            when {
                // Skip build if we're just deploying an existing tag
                not { expression { return params.IMAGE_TAG != 'latest' } }
            }
            steps {
                sh 'mvn clean package'
            }
        }

        stage('Test') {
            when {
                not { expression { return params.SKIP_TESTS } }
            }
            steps {
                sh 'mvn test'
```

```
            }
        }

        stage('Deploy') {
            steps {
                script {
                    def helmCmd = """
                        helm upgrade --install myapp ./chart \
                            --namespace ${params.TARGET_ENV} \
                            --set image.tag=${params.IMAGE_TAG} \
                            --set logLevel=${params.LOG_LEVEL} \
                            -f values-${params.TARGET_ENV}.yaml \
                            --wait \
                            ${HELM_DRY_RUN}
                    """

                    if (params.DRY_RUN) {
                        echo "=== DRY RUN - Command that would execute:
===="
                        echo helmCmd
                    } else {
                        sh helmCmd
                    }
                }
            }
        }
    }
}
```

⚙️ Advanced Parameter Patterns

Pattern 1: Active Choices (Dynamic Parameters)

```
// Active Choices Plugin - parameter options computed dynamically
// Requires: uno-choice plugin

properties([
    parameters([
        // Parameter whose choices are dynamically generated by a Groovy
script:
        [$class: 'ChoiceParameter',
            choiceType: 'PT_SINGLE_SELECT',
            description: 'Select version to deploy',
            name: 'APP_VERSION',
            script: [$class: 'GroovyScript',
                script: [classpath: [], sandbox: true,
                    script: '''
                        // Dynamically fetch available versions from Nexus
                        def url =
"https://nexus.example.com/service/rest/v1/search/assets?
```

```
repository=releases&name=myapp"
                        def response = new URL(url).text
                        def json = new
groovy.json.JsonSlurper().parseText(response)
                        return json.items.collect { it.version
}.unique().sort().reverse()
                    '''
                ]
            ]
        ],

        // Reactive parameter that updates based on APP_VERSION selection:
        [$class: 'CascadeChoiceParameter',
            choiceType: 'PT_SINGLE_SELECT',
            description: 'Target environment for this version',
            name: 'TARGET_ENV',
            referencedParameters: 'APP_VERSION',
            script: [$class: 'GroovyScript',
                script: [classpath: [], sandbox: true,
                    script: '''
                        // If version contains 'SNAPSHOT', only allow non-
production
                        if (APP_VERSION.contains('SNAPSHOT')) {
                            return ['dev', 'staging']
                        }
                        return ['dev', 'staging', 'production']
                    '''
                ]
            ]
        ]
    ])
])
```

Pattern 2: Parameters from Pipeline (Programmatic Triggering)

```
// Triggering a parameterized pipeline from another pipeline:
stage('Trigger Deploy') {
    steps {
        build job: 'deploy-pipeline',
              wait: true,
              parameters: [
                  string(name: 'IMAGE_TAG',   value:
"${env.BUILD_NUMBER}-${env.GIT_COMMIT[0..6]}"),
                  string(name: 'TARGET_ENV',  value: 'staging'),
                  booleanParam(name: 'SKIP_TESTS', value: false),
                  choice(name: 'LOG_LEVEL',   value: 'INFO')
              ]
    }
}

// Triggering from shell/API (for external systems):
```

```
// curl -X POST https://jenkins.example.com/job/deploy-
pipeline/buildWithParameters \
//   --data "IMAGE_TAG=1.2.3-
abc1234&TARGET_ENV=production&SKIP_TESTS=false" \
//   -u user:token
```

Pattern 3: Parameter-Driven Stage Matrix (Before 3.14)

```
// Before matrix builds existed, parameters drove multi-environment
testing:
pipeline {
    agent none
    parameters {
        choice(name: 'JAVA_VERSION', choices: ['11', '17', '21'])
        choice(name: 'OS',           choices: ['ubuntu', 'centos',
'alpine'])
    }
    stages {
        stage('Test') {
            agent {
                docker { image "eclipse-temurin:${params.JAVA_VERSION}-
jdk-${params.OS}" }
            }
            steps {
                sh "java -version && mvn test"
            }
        }
    }
}
```

🔑 Key Concepts

Concept	Meaning
`params.NAME`	Access a parameter value anywhere in the pipeline
`defaultValue`	Value used when no value is provided — always set a safe default
First-run bootstrap	On first encounter of `parameters{}`, Jenkins reads them but doesn't apply them until second run
Parameter validation	Check params early in a `script{}` block — fail fast before doing expensive work
`choice` default	First item in the `choices` list is the default — order intentionally
`password` masking	Password parameters are masked in logs — never echoed to console

Concept	Meaning
`booleanParam` access	`params.SKIP_TESTS` is a Java `Boolean` — use in `if (params.SKIP_TESTS)` directly

💬 Short Crisp Interview Answer

> *"Pipeline parameters are defined in a `parameters {}` block and make pipelines flexible and reusable. Types: `string` for free text, `choice` for dropdowns, `booleanParam` for checkboxes, `text` for multiline, `password` for masked secrets. Parameters are accessed as `params.PARAMETER_NAME` anywhere in the pipeline. Best practices: always validate parameters early — check before doing expensive work; always provide safe `defaultValue`s; never use `defaultValue: 'production'` for environment targeting; and for `choice` parameters, put the safest/most common option first since it's the default. The first-run bootstrap gotcha: when you add a `parameters{}` block to an existing Jenkinsfile, the very first build after the push doesn't get the parameters — Jenkins discovers them on that first run, and subsequent builds use them."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **First-run bootstrap.** Adding `parameters {}` to a Jenkinsfile for the first time: the first triggered build runs WITHOUT those parameters (Jenkins hasn't read the new Jenkinsfile yet). Always set safe `defaultValue`s for every parameter — the first build will use them without user input.
- **`choice` parameter first item is always the default.** You cannot programmatically set a different default for choices — the first item in the list IS the default. Put the safest or most common option first (`dev` before `production`).
- **`booleanParam` default `true` is dangerous.** `booleanParam(name: 'SKIP_TESTS', defaultValue: true)` means automated builds (webhooks, cron) skip tests by default. Always default booleans to the safe conservative value (`false`).
- **`password` parameter values ARE passed to downstream jobs** if you use `build job: '...', parameters: [password(...)]`. They're masked in logs but still transmitted in build records. Prefer credential binding for secrets over `password` parameters.
- **Parameters block requires a `properties([parameters([...])])` wrapper in Scripted Pipeline.** The `parameters {}` directive is Declarative-only. In Scripted Pipeline: `properties([parameters([string(name: 'TAG', defaultValue: 'latest')])])`.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Declarative Pipeline (3.2)	`parameters {}` is a Declarative directive
Pipeline Triggers (3.9)	Parameterized builds triggered via API, upstream jobs
Credentials (3.12)	Use credentials binding for secrets, not `password` parameters
Matrix Builds (3.14)	Matrix supersedes manual parameter-driven multi-axis builds

Topic 3.9 — Pipeline Triggers

🟡 Intermediate | Automated Build Initiation

📌 What It Is — In Simple Terms

Pipeline Triggers define the conditions under which a pipeline starts automatically — without a human clicking "Build Now." Jenkins supports several trigger types: time-based schedules (cron), SCM change detection (polling or webhooks), upstream job completion, and HTTP-triggered builds.

Getting triggers right is critical for CI/CD: wrong triggers mean either builds don't start when they should, or they start too often, wasting resources.

⚙️ All Trigger Types — Complete Reference

```
pipeline {
    agent any

    triggers {

        // ── CRON: time-based schedule ──────────────────────────────
        // Standard cron syntax: MINUTE HOUR DOM MONTH DOW
        // H = "hash" — distributes load, not exact time
        cron('H 2 * * 1-5')         // ~2am weekdays (exact minute varies
by job hash)
        cron('H/15 * * * *')        // Every ~15 minutes
        cron('H H(9-16)/2 * * 1-5') // Every 2 hours during business
hours, weekdays
        cron('@daily')               // Once a day (midnight)
        cron('@weekly')              // Once a week (Sunday midnight)

        // ── SCM POLLING: check SCM for changes on a schedule ──────
        // AVOID in production — use webhooks instead
        // Polling runs FROM the Controller, creating network + CPU load
        pollSCM('H/5 * * * *')      // Check every ~5 minutes for changes
        pollSCM('H H(7-18) * * 1-5') // Poll only during business hours

        // ── GITHUB PUSH: webhook-based trigger ────────────────────
        // Requires: GitHub plugin installed
        // Jenkins receives POST from GitHub on push/PR events
        githubPush()

        // ── UPSTREAM JOB: trigger when another job succeeds ───────
        upstream(
            upstreamProjects: 'myapp-build-pipeline',
            threshold: hudson.model.Result.SUCCESS  // Only on SUCCESS
            // threshold options: SUCCESS, UNSTABLE, FAILURE, NOT_BUILT
        )
```

```
        // Multiple upstream projects:
        upstream(
            upstreamProjects: 'build-pipeline,integration-test-pipeline',
            threshold: hudson.model.Result.SUCCESS
        )

        // ── GENERIC WEBHOOK TRIGGER ──────────────────────────────────
        // Generic Webhook Trigger plugin — handles any webhook payload
        GenericTrigger(
            genericVariables: [
                [key: 'BRANCH',    value: '$.ref'],           // Extract
from JSON payload
                [key: 'COMMITTER', value: '$.pusher.name'],
                [key: 'COMMIT_ID', value: '$.head_commit.id']
            ],
            causeString:       'Triggered by push to $BRANCH',
            token:             'my-secret-webhook-token',    // Security
token in webhook URL
            printContributedVariables: true,
            printPostContent:  false,                        // Don't log
payload (security)
            regexpFilterText:  '$BRANCH',
            regexpFilterExpression: '^refs/heads/main$'      // Only
trigger on main branch pushes
        )
    }

    stages {
        stage('Build') {
            steps {
                echo "Triggered by: ${currentBuild.getBuildCauses()
[0].shortDescription}"
                sh 'mvn clean package'
            }
        }
    }
}
```

⚙️ Webhook Configuration — The Right Way

GitHub Webhook Setup

```
1. GitHub Repository → Settings → Webhooks → Add webhook

Payload URL: https://jenkins.example.com/github-webhook/
Content type: application/json
Secret: (optional but recommended — validates payload authenticity)

Events to trigger on:
  ✅ Push events
```

```
  ✅ Pull request events (for PR builds)

Jenkins side:
  - Install GitHub plugin
  - Add `githubPush()` trigger to pipeline
  - OR: Multibranch Pipeline auto-configures webhooks

Jenkins requirement:
  - Jenkins URL must be publicly reachable from GitHub
  - For private Jenkins: use GitHub Enterprise or a webhook relay
```

GitLab Webhook Setup

```
1. GitLab Project → Settings → Webhooks

URL: https://jenkins.example.com/project/my-pipeline
     (or: https://jenkins.example.com/multibranch-webhook-trigger/invoke?
token=TOKEN)

Secret Token: <generate random token>
Trigger: Push events, Merge request events

Jenkins side:
  - Install GitLab plugin
  triggers {
      gitlab(
          triggerOnPush: true,
          triggerOnMergeRequest: true,
          branchFilterType: 'All',
          secretToken: "${env.GITLAB_WEBHOOK_TOKEN}"
      )
  }
```

Generic Webhook for Any SCM / External System

```
# Generic Webhook Trigger plugin — trigger from anything
# URL pattern: https://jenkins.example.com/generic-webhook-trigger/invoke?
token=TOKEN

# Trigger from shell:
curl -X POST \
  "https://jenkins.example.com/generic-webhook-trigger/invoke?token=my-
secret-token" \
  -H "Content-Type: application/json" \
  -d '{"branch": "main", "version": "1.2.3", "environment": "staging"}'

# Jenkins extracts variables from payload via JSONPath:
triggers {
```

```
    GenericTrigger(
        genericVariables: [
            [key: 'VERSION',     value: '$.version'],
            [key: 'TARGET_ENV',  value: '$.environment'],
            [key: 'BRANCH',      value: '$.branch']
        ],
        token: 'my-secret-token',
        printPostContent: false
    )
}
// Variables available in pipeline as env.VERSION, env.TARGET_ENV,
env.BRANCH
```

⚙️ The Cron "H" Syntax — In Detail

```
Standard cron: 0/5 * * * *    → runs at EXACTLY :00, :05, :10, :15...
Jenkins H syntax: H/5 * * * * → distributes across jobs to avoid
thundering herd

How H works:
  H = hash of the job name
  For job "auth-service": hash('auth-service') % 5 = 2 → runs at :02, :07,
:12...
  For job "order-service": hash('order-service') % 5 = 4 → runs at :04,
:09, :14...

  Both configured "H/5" but run at different minutes
  → Load spread across the 5-minute window
  → No thundering herd at :00 mark

Common patterns:
  H/5  * * * *          Every ~5 minutes (polling)
  H    2 * * *          Once a day, around 2am
  H    2 * * 1-5        Once on weekdays, around 2am
  H(0-30) 2 * * *       Daily, some time between 2:00-2:30am
  H/30 * * * *          Every ~30 minutes
  H H  * * *            Once a day, at a consistent but unpredictable time
  H H  * * 0            Once a week on Sunday
```

⚙️ Trigger Best Practices

```
// ── PRODUCTION PATTERN: webhooks + cron backup ──────────────────
triggers {
    // Primary: GitHub webhook → builds immediately on push
    githubPush()

    // Safety net: daily build even with no code changes
```

```
        // Catches: dependency updates, environment drift, scheduled reports
        cron('H 3 * * *')
    }

    // ── SCHEDULED JOBS ONLY: no webhook ──────────────────────────────
    // For jobs that run on schedule (reports, backups, dependency scans):
    triggers {
        cron('H 2 * * 1')  // Every Monday ~2am — weekly dependency scan
    }

    // ── UPSTREAM TRIGGER WITH FILTERING ──────────────────────────────
    // Deploy pipeline: only triggers when build pipeline succeeds AND
    // it was a main branch build:
    triggers {
        upstream(
            upstreamProjects: 'myapp-build',
            threshold: hudson.model.Result.SUCCESS
        )
    }
    // Add additional when{} condition to filter by branch:
    stages {
        stage('Deploy') {
            when {
                // Only auto-deploy if triggered by upstream (not manual)
                triggeredBy cause: 'UpstreamCause'
                // AND upstream build was on main:
                expression {
                    def cause =
currentBuild.getBuildCauses('hudson.model.Cause$UpstreamCause')[0]
                    return cause?.upstreamProject?.endsWith('/main')
                }
            }
        }
    }
```

🔑 Key Concepts

Concept	Meaning
`cron()`	Time-based trigger — runs on a schedule regardless of code changes
`pollSCM()`	Controller polls Git on schedule — avoid, use webhooks
`githubPush()`	GitHub pushes webhook to Jenkins — immediate trigger
`upstream()`	Triggers when another job completes with specified result
`GenericTrigger`	Webhook trigger plugin that extracts variables from any JSON payload
`H` syntax	Hash-based cron — distributes jobs to avoid thundering herd

Concept	Meaning
`currentBuild.getBuildCauses()`	Programmatically detect what triggered this build
Webhook vs polling	Webhooks: push (instant, no load). Polling: pull (delayed, creates load)

💬 Short Crisp Interview Answer

> *"Jenkins has four main trigger types: `cron` for scheduled builds, `pollSCM` for SCM polling (which I avoid — it creates unnecessary load and latency), `githubPush` or equivalent webhook triggers for instant builds on code push, and `upstream` to trigger when another job succeeds. In production, I always use webhooks over polling: webhooks are push-based — GitHub notifies Jenkins immediately on push, no load on the Controller, sub-second trigger latency. Polling is pull-based — the Controller makes outbound Git requests on a schedule, creates CPU and network load, and has up to the polling interval of latency. For cron schedules, I always use the `H` (hash) syntax instead of exact times — `H 2 * * *` distributes jobs randomly within the hour to prevent thundering herd at midnight."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **`triggers {}` in Multibranch Pipelines need special handling.** In a Multibranch Pipeline, `triggers { githubPush() }` in the Jenkinsfile doesn't work the same way — the Multibranch job itself has a webhook that fires branch indexing. You configure the webhook at the Multibranch folder level, not in each branch's Jenkinsfile.
- **`pollSCM` and `cron` triggers are only active after the FIRST build.** A brand-new pipeline job has no triggers active until it has run at least once and Jenkins has read the Jenkinsfile. This is the "cold start" problem for scheduled jobs — manually run once after creating the job.
- **`upstream()` trigger threshold.** `hudson.model.Result.SUCCESS` is the class reference, not a string. Common mistake: using `'SUCCESS'` (string) instead of `hudson.model.Result.SUCCESS`. The string silently fails to match.
- **Webhook secrets.** Configure a webhook secret in GitHub and validate it in Jenkins. Without a secret, anyone who knows your Jenkins webhook URL can trigger builds with arbitrary payloads — a potential security and resource abuse vector.
- **Jenkins URL must be reachable from GitHub/GitLab.** If Jenkins is behind a corporate firewall, GitHub webhooks can't reach it. Solutions: use ngrok for local dev, a reverse proxy / API gateway for production, GitHub Enterprise Server inside the network, or a webhook relay service.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Controller Internals (2.3)	SCM polling creates load on Controller — topic 2.3 explains the thread model
Jenkinsfile (3.5)	`triggers {}` block lives in Jenkinsfile

Connected Topic	How It Connects
Multi-Branch Pipelines (4.1)	Multibranch handles webhooks differently from single Pipeline jobs
Jenkins at Scale (2.8)	Webhook vs polling has major impact at scale

---

Topic 3.10 — Stash and Unstash ⚠

🟡 Intermediate | Cross-Stage File Transfer

📌 What It Is — In Simple Terms

Stash copies files from an agent's workspace to the Jenkins Controller's temporary storage. **Unstash** retrieves those files to a different agent's workspace. Together, they solve the problem of passing build outputs between pipeline stages that run on different agents.

The confusion stash/unstash causes in interviews comes from misunderstanding when it's needed versus when files are already accessible.

```
Without stash (WRONG — file not accessible on different agent):
  Stage 1 on Agent-A: sh 'mvn package' → creates target/myapp.jar
  Stage 2 on Agent-B: sh 'docker build .' → ERROR: target/myapp.jar NOT
HERE

With stash (CORRECT):
  Stage 1 on Agent-A: sh 'mvn package'
                       stash name: 'jar', includes: 'target/*.jar'
  Stage 2 on Agent-B: unstash 'jar'          ← copies jar from Controller
                       sh 'docker build .'    ← jar is now present ✅
```

🔍 Why Stash Exists — The Workspace Problem

```
Each agent has its OWN filesystem and workspace directory.
Agent-A workspace: /home/jenkins/workspace/myapp-main/
Agent-B workspace: /home/jenkins/workspace/myapp-main/

These are DIFFERENT directories on DIFFERENT machines.
A file created on Agent-A does NOT exist on Agent-B.

Stash/Unstash works by routing through the Controller:
  Agent-A → [tar.gz of files] → Controller (tmp storage) → Agent-B
                                        ↑
                                Stored in JENKINS_HOME/caches/
                                (or temp dir, depending on version)
```

⚙️ Complete Stash/Unstash Reference

```
pipeline {
    agent none   // Different agents per stage — this is when stash is
needed

    stages {

        // ── STAGE 1: BUILD (creates artifacts) ──────────────────────
        stage('Build') {
            agent { label 'java-build' }
            steps {
                checkout scm
                sh 'mvn clean package -DskipTests'

                // ── BASIC STASH ──────────────────────────────────
                stash(
                    name:     'compiled-artifacts',   // Unique name —
used to unstash later
                    includes: 'target/*.jar,target/*.war',   // Glob
patterns (relative to workspace)
                    excludes: 'target/*-sources.jar',        // Exclude
source jars
                    allowEmpty: false                         // Fail if
nothing matches
                )

                // ── STASH MULTIPLE NAMED BUNDLES ─────────────────
                stash name: 'test-config',
                      includes: 'src/test/resources/**,test-data/**'

                stash name: 'helm-chart',
                      includes: 'chart/**,values-*.yaml'

                stash name: 'docker-context',
                      includes: 'Dockerfile,target/*.jar,config/**'
            }
        }

        // ── STAGE 2: TEST (uses artifacts, runs on different agent) ──
        stage('Integration Test') {
            agent { label 'test-runner' }   // Different agent than 'java-
build'
            steps {
                // Get the JAR built in Stage 1
                unstash 'compiled-artifacts'    // Restores to same
relative path
                unstash 'test-config'

                // Run integration tests using the stashed JAR
```

```
                sh '''
                    java -jar target/myapp.jar &
                    sleep 10
                    mvn verify -Pintegration-tests
                '''
            }
            post {
                always {
                    junit 'target/failsafe-reports/*.xml'
                    // Stash test results to be collected later
                    stash name: 'test-results',
                          includes: 'target/failsafe-
reports/**,target/site/**',
                          allowEmpty: true
                }
            }
        }

        // ── STAGE 3: PARALLEL STAGES using same stash ──────────────────
        stage('Quality Gates') {
            parallel {
                stage('SAST') {
                    agent { label 'security' }
                    steps {
                        unstash 'compiled-artifacts'  // Both parallel
stages
                        sh 'semgrep --config auto .'  // can unstash same
bundle
                    }
                }
                stage('Container Build') {
                    agent { label 'docker' }
                    steps {
                        unstash 'docker-context'      // Different bundle
                        sh "docker build -t myapp:${BUILD_NUMBER} ."
                        sh "docker push myapp:${BUILD_NUMBER}"
                    }
                }
            }
        }

        // ── STAGE 4: DEPLOY (uses helm chart stash) ────────────────────
        stage('Deploy') {
            agent { label 'deploy' }
            steps {
                unstash 'helm-chart'
                sh """
                    helm upgrade --install myapp ./chart \
                        --set image.tag=${BUILD_NUMBER} \
                        -f values-staging.yaml
                """
            }
        }
```

```
    }
}
```

⚙️ Stash Limitations and Alternatives

```
Stash Limitations:
  1. SIZE LIMIT: 100MB default per stash
     Large stashes (fat JARs, test data, Docker contexts) are slow or fail
     Configure: -
Dorg.jenkinsci.plugins.workflow.support.pickles.serialization.RiverWriter.
MAXIMUM_SIZE=200mb

  2. TEMPORARY: Stashes exist only for the DURATION OF THE BUILD
     Cannot be accessed in a future build — use archiveArtifacts or
artifact repo instead

  3. CONTROLLER BANDWIDTH: All stash traffic passes through Controller
     With 50 parallel stages all stashing 50MB = 2.5GB through Controller
     Creates I/O bottleneck on Controller

  4. NO DEDUPLICATION: Stashing same file in multiple bundles wastes space
```

```
// ── ALTERNATIVES TO STASH ──────────────────────────────────────────

// Alternative 1: Same agent for all stages (workspace naturally shared)
pipeline {
    agent { label 'linux' }    // ALL stages run on SAME agent
    stages {
        stage('Build') {
            steps { sh 'mvn package' }          // creates
target/myapp.jar
        }
        stage('Test') {
            steps { sh 'mvn verify' }            // target/myapp.jar
accessible ✅
        }
        stage('Docker') {
            steps { sh 'docker build -t myapp .' }  // ✅ same workspace
        }
    }
}
// When to use: simple pipelines where all stages can use same build
environment

// Alternative 2: Artifact repository (for large files or cross-build
sharing)
stage('Build') {
    steps {
```

```
        sh 'mvn clean deploy'   // Push to Nexus/Artifactory
        // OR:
        sh 'docker build -t registry.io/myapp:${BUILD_NUMBER} . && docker
push registry.io/myapp:${BUILD_NUMBER}'
    }
}
stage('Deploy') {
    steps {
        // Pull from registry - not from stash
        sh 'helm upgrade --install myapp ./chart --set
image.tag=${BUILD_NUMBER}'
    }
}
// When to use: artifacts >100MB, need to persist beyond build, cross-
pipeline sharing

// Alternative 3: Shared PVC (Kubernetes agents)
// All pods in the same build share an emptyDir or PVC mounted at same
path
// Files written by maven container readable by docker container
// See Topic 2.5 for full example

// Alternative 4: archiveArtifacts (persists to Jenkins build record)
stage('Build') {
    steps {
        sh 'mvn package'
        archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
    }
}
stage('Deploy') {
    steps {
        // Copy artifact from previous build record:
        copyArtifacts(
            projectName:  env.JOB_NAME,
            selector:     specific(env.BUILD_NUMBER),   // This build's
archived artifacts
            filter:       'target/*.jar'
        )
        sh 'deploy.sh target/*.jar'
    }
}
```

🔑 Key Concepts

Concept	Meaning
`stash name: 'N'`	Archive files to Controller temp storage, named `N`
`unstash 'N'`	Retrieve stash named `N` to current workspace
`includes`	Glob pattern of files to include (relative to workspace root)

Concept	Meaning
`excludes`	Glob pattern of files to exclude from stash
`allowEmpty: true`	Don't fail if no files match the includes pattern
Stash scope	Exists only for the current build — not accessible cross-build
100MB limit	Default size limit per stash bundle
Controller routing	All stash traffic goes through Controller (potential bottleneck)
Named stashes	Multiple stashes can coexist — use descriptive names

---

💬 Short Crisp Interview Answer

> *"Stash and unstash solve the problem of sharing files between pipeline stages that run on different agents. Each agent has its own filesystem, so a JAR built on Agent-A isn't accessible on Agent-B. `stash` archives files to the Jenkins Controller's temporary storage; `unstash` retrieves them to any agent's workspace. The key constraints: 100MB default size limit per stash, stashes only persist for the current build (not accessible in future builds), and all traffic routes through the Controller. For large files over 100MB — Docker images, test data, fat JARs — use an artifact repository like Nexus or the Docker registry instead of stash. For same-build artifact sharing without stash, the simplest approach is running all stages on the same agent, where the workspace is naturally shared. For Kubernetes pod agents, mount an emptyDir volume shared between all containers in the pod."*

🔬 Deep Dive Answer

When stash routes through Controller vs direct agent transfer:

```
In all Jenkins versions through LTS 2.x:
  stash ALWAYS routes through Controller JVM

  Agent-A → [tar.gz] → Controller RAM/disk → [tar.gz] → Agent-B

  With 10 stages each stashing 50MB:
    10 × 50MB = 500MB through Controller
    Controller I/O becomes bottleneck

  Mitigation:
    1. Stash only what you need (minimal includes patterns)
    2. Prefer same-agent execution when possible
    3. Use artifact repos for large files
    4. Pipeline: PERFORMANCE_OPTIMIZED durability reduces other I/O
pressure
    5. Jenkins 2.401+ has "direct agent-to-agent" artifact transfer
experiment
```

Stash With Conditional Unstash:

```
stage('Deploy') {
    steps {
        script {
            // Only unstash if the stash was created (might not exist if
build was skipped)
            try {
                unstash 'helm-chart'
            } catch (Exception e) {
                echo "No stashed helm chart — using chart from repository"
                checkout scm
            }
            sh 'helm upgrade --install myapp ./chart'
        }
    }
}
```

🏭 Real-World Production Example

Airbnb's Build Pipeline Architecture: Airbnb's Java service pipeline uses distinct agents for each stage (for cost efficiency — build on beefy agents, deploy on minimal agents). The flow: compile on 8-CPU Maven agent → stash JAR (12MB) → run integration tests on test agent (unstash JAR, start server) → stash test results → docker build on Docker agent (unstash JAR) → deploy on k8s agent (uses image tag from registry, no stash needed). Key insight: they ONLY stash the JAR for the integration test stage. The Docker build stage pulls from the registry after the image is built — not stash. This avoids a 500MB+ Docker context going through the Controller.

❓ Common Interview Questions & Strong Answers

Q1: ⚠ When would you use stash vs archiveArtifacts vs a Docker registry?

> `stash`: short-lived, within the same build, files needed by a later stage on a different agent. Limit: 100MB, exists only for this build. Use for: compiled JARs needed by integration tests, test configs, generated code. `archiveArtifacts`: persists the file to the Jenkins build record — accessible in Jenkins UI, can be downloaded, survives pipeline completion. Use for: test reports, generated documentation, build artifacts you want to reference later. Docker registry: for container images — the right artifact store for anything containerized. Don't stash or archive Docker images (they're hundreds of MB to GB). For anything over 100MB or needed across builds, use an artifact repository (Nexus, Artifactory) or object storage (S3).

Q2: A pipeline has 5 parallel stages after a build stage. All 5 need the compiled JAR. How do you handle this?

> Stash the JAR once after the build stage with `stash name: 'compiled-jar', includes: 'target/*.jar'`. All 5 parallel stages call `unstash 'compiled-jar'` at the start. Each parallel stage gets its own copy in its own workspace. One concern: 5 × [JAR size] of data through the Controller simultaneously. If the JAR is 50MB, that's 250MB of Controller I/O for just this one build. If many builds run concurrently, this multiplies. If it becomes a bottleneck, the alternative is to push the

> JAR to Nexus in the build stage and have each parallel stage pull from Nexus — distributing the download load rather than concentrating it on the Controller.

⚠ Tricky Edge Cases & Gotchas

- ⚠ **Stash `includes` patterns are relative to workspace root, not to a subdirectory.** `stash name: 'jar', includes: '*.jar'` looks for JARs AT the workspace root, not recursively. For `target/myapp.jar`, you need `includes: 'target/*.jar'` or `includes: '**/*.jar'`.
- **Unstash overwrites files at the same relative path.** If `target/myapp.jar` exists in Agent-B's workspace before `unstash`, it is overwritten without warning. This is usually the desired behavior but can surprise when you have different build outputs with the same name.
- **Multiple `unstash` calls with same name are additive.** Calling `unstash 'config'` twice in the same stage adds the files twice (the second call has no new files to add since they already exist). It's not an error but is wasteful.
- **Stash names must be unique within a build.** Calling `stash name: 'artifacts'` twice in the same build does NOT create two stashes — the second call overwrites the first. Name stashes meaningfully and uniquely.
- **`allowEmpty: false` (default) fails if `includes` matches nothing.** If your build conditionally produces artifacts and you stash unconditionally, the build fails when no JAR is produced. Use `allowEmpty: true` for optional files and check if they exist before unstashing.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Parallel Stages (3.7)	Parallel stages commonly need stash to receive shared artifacts
Workspaces (2.5)	Stash solves the cross-workspace isolation problem
Agent Types (2.4)	Kubernetes emptyDir volumes replace stash within a pod
Jenkins at Scale (2.8)	Stash traffic through Controller is a scaling concern
Artifact Management (4.3)	Artifact repositories are the production alternative to stash

---

Topic 3.11 — Environment Variables

🟡 Intermediate | Context and Configuration

📌 What It Is — In Simple Terms

Environment variables in Jenkins pipelines are key-value pairs accessible throughout the pipeline execution — in shell commands, in Groovy code, and in step configurations. Jenkins provides many built-in variables automatically, you can define custom ones, and credentials can be injected as environment variables securely.

Understanding which variables exist, how they're scoped, and how secrets are handled safely is essential for production pipeline work.

⚙️ Built-In Jenkins Environment Variables

```
// ALL BUILT-IN ENV VARS — always available in every pipeline

pipeline {
    agent any
    stages {
        stage('Show Built-Ins') {
            steps {
                script {
                    // ── JOB IDENTITY ──────────────────────────────
                    echo "Job name:          ${env.JOB_NAME}"          //
myapp/main (folder/job)
                    echo "Job base name:     ${env.JOB_BASE_NAME}"     //
main (just the leaf name)
                    echo "Job URL:           ${env.JOB_URL}"           //
https://jenkins.../job/myapp/job/main/
                    echo "Build number:      ${env.BUILD_NUMBER}"      //
42
                    echo "Build ID:          ${env.BUILD_ID}"          //
42 (same as BUILD_NUMBER in modern Jenkins)
                    echo "Build URL:         ${env.BUILD_URL}"         //
https://jenkins.../job/myapp/42/
                    echo "Build tag:         ${env.BUILD_TAG}"         //
jenkins-myapp-main-42
                    echo "Display name:      ${env.BUILD_DISPLAY_NAME}" //
#42

                    // ── JENKINS SYSTEM ────────────────────────────
                    echo "Jenkins URL:       ${env.JENKINS_URL}"       //
https://jenkins.example.com/
                    echo "Jenkins home:      ${env.JENKINS_HOME}"      //
/var/jenkins_home
                    echo "Node name:         ${env.NODE_NAME}"         //
agent-linux-1 (or 'master')
                    echo "Node labels:       ${env.NODE_LABELS}"       //
linux docker java maven
                    echo "Executor number:   ${env.EXECUTOR_NUMBER}"   //
0 (which executor slot)
                    echo "Workspace:         ${env.WORKSPACE}"         //
/home/jenkins/workspace/myapp/main

                    // ── SCM / GIT ─────────────────────────────────
                    echo "Branch name:       ${env.BRANCH_NAME}"       //
main (Multibranch only)
                    echo "Branch is PR:      ${env.CHANGE_ID}"         //
PR number if this is a PR build
                    echo "Change target:     ${env.CHANGE_TARGET}"     //
```

```
target branch of PR
                    echo "Change author:     ${env.CHANGE_AUTHOR}"     //
GitHub username of PR author
                    echo "Change title:      ${env.CHANGE_TITLE}"      //
PR title
                    echo "Git commit:        ${env.GIT_COMMIT}"        //
full SHA: abc123def456...
                    echo "Git branch:        ${env.GIT_BRANCH}"        //
origin/main
                    echo "Git URL:           ${env.GIT_URL}"           //
https://github.com/org/repo
                    echo "Git previous:      ${env.GIT_PREVIOUS_COMMIT}"
// previous build's SHA

                    // ── PIPELINE STATE ──────────────────────────
                    echo "Build result:      ${currentBuild.result}"
// null (not yet determined)
                    echo "Current result:
${currentBuild.currentResult}"  // SUCCESS (current state)
                    echo "Previous result:
${currentBuild.previousBuild?.result}" // last build result
                    echo "Duration (ms):     ${currentBuild.duration}"
// milliseconds
                    echo "Duration string:
${currentBuild.durationString}" // "1 min 23 sec"
                    echo "Build causes:
${currentBuild.getBuildCauses()}"
                }

                // In shell scripts, use $ directly:
                sh '''
                    echo "Job: ${JOB_NAME}, Build: ${BUILD_NUMBER}"
                    echo "Git SHA: ${GIT_COMMIT}"
                    echo "Branch: ${BRANCH_NAME}"
                    echo "Workspace: ${WORKSPACE}"
                '''
            }
        }
    }
}
```

⚙️ Custom Environment Variables — All Scopes

```
pipeline {
    agent any

    // ── PIPELINE-WIDE ENVIRONMENT ──────────────────────────────────
    // Available in ALL stages
    environment {
        APP_NAME    = 'myservice'
```

```
        REGISTRY    = 'registry.example.com/myorg'
        APP_VERSION = sh(script: 'cat VERSION', returnStdout: true).trim()
// Shell command result

        // String interpolation with built-ins:
        IMAGE_TAG   = "${APP_NAME}:${BUILD_NUMBER}-${GIT_COMMIT.take(7)}"
        FULL_IMAGE  = "${REGISTRY}/${IMAGE_TAG}"

        // Boolean-like:
        IS_MAIN_BRANCH = "${env.BRANCH_NAME == 'main'}"
        // Access as: if (env.IS_MAIN_BRANCH == 'true') {...}

        // Credential injection (secure):
        DOCKER_CREDS  = credentials('docker-registry-creds')
        // Expands to: DOCKER_CREDS_USR + DOCKER_CREDS_PSW +
DOCKER_CREDS=usr:pass
        SONAR_TOKEN   = credentials('sonarqube-token')
        // Single string secret: SONAR_TOKEN = the secret value
    }

    stages {
        stage('Build') {
            // ── STAGE-SCOPED ENVIRONMENT ──────────────────────────────
            // Overrides pipeline-level for THIS STAGE ONLY
            environment {
                BUILD_OPTS = '-Dmaven.test.skip=false -
Dcheckstyle.skip=false'
                MAVEN_OPTS = '-Xmx1g -XX:+TieredCompilation'
            }
            steps {
                sh 'mvn clean package ${BUILD_OPTS}'
                // BUILD_OPTS and MAVEN_OPTS are available here
                // APP_NAME from pipeline-level is also available
            }
            // After this stage, BUILD_OPTS and MAVEN_OPTS are no longer
in scope
        }

        stage('Deploy') {
            environment {
                TARGET_NS = "${params.TARGET_ENV ?: 'staging'}"
                REPLICA_COUNT = "${env.BRANCH_NAME == 'main' ? '3' : '1'}"
            }
            steps {
                sh """
                    helm upgrade --install ${APP_NAME} ./chart \
                        --namespace ${TARGET_NS} \
                        --set replicaCount=${REPLICA_COUNT} \
                        --set
image.tag=${BUILD_NUMBER}-${GIT_COMMIT.take(7)}
                """
            }
        }
```

```
        stage('Dynamic Env Vars in script{}') {
            steps {
                script {
                    // Set env vars dynamically in Groovy
                    env.DYNAMIC_VAR = sh(script: 'git describe --tags --
abbrev=0',
                                         returnStdout: true).trim()

                    env.DEPLOY_TIME  = new Date().format('yyyy-MM-dd
HH:mm:ss')
                    env.BUILD_ACTOR  = currentBuild.getBuildCauses()
[0]?.userId ?: 'automation'

                    // These are now available in subsequent stages too
                    echo "Deploying version: ${env.DYNAMIC_VAR}"
                }
            }
        }
    }
}
```

⚙️ GString Interpolation vs Shell Dollar Signs

```
// CRITICAL: Groovy GString vs shell variable — different $ behavior

stage('Dollar Sign Rules') {
    steps {
        // ── GROOVY GSTRING (double quotes "...") ──────────────────────
        // ${} is GROOVY interpolation — evaluated BEFORE shell runs
        sh "echo Hello, ${env.BRANCH_NAME}"
        // Shell actually runs: echo Hello, main
        // ✅ Use for: injecting Jenkins env vars into shell commands

        // ── SHELL VARIABLE (single quotes '...' or escaped \$) ────────
        // \$ or $var inside shell script is evaluated BY THE SHELL
        sh 'echo "Home: $HOME, Path: $PATH"'
        // ✅ Use for: shell variables set within the script, not Jenkins
vars

        // ── MIXED (multi-line with both) ──────────────────────────────
        sh """
            export IMAGE=${env.IMAGE_TAG}    # Jenkins var injected here
            docker build -t \$IMAGE .        # Shell var used here
(escaped \$)
            echo "Built: \$(docker images | grep myapp | awk '{print
\$2}')"
        """
        // Inside """ """: ${env.X} → Groovy interpolation (before shell)
        //                 \$X or '$X' → shell variable (evaluated by
shell)
```

```
        //                  \$(...) → shell subshell (evaluated by shell)

        // ── TRIPLE SINGLE QUOTES (no Groovy interpolation at all) ────
        sh '''
            VERSION=$(cat VERSION)           # Shell subshell — works
            IMAGE="myapp:${VERSION}"         # Shell variable substitution
— works
            echo "Building: $IMAGE"
        '''
        // ✅ Use for: multi-line shell scripts with shell variables and
subshells
        // Jenkins env vars NOT accessible here — must set them before
this block

        // ── SAFE PATTERN: inject Jenkins vars, then use shell vars ────
        def imageTag = env.IMAGE_TAG   // Capture in Groovy first
        sh """
            TAG="${imageTag}"            # Now it's a shell variable
            docker build -t \$TAG .      # Use shell variable
            docker tag \$TAG registry/myapp:\$TAG
        """
    }
}
```

⚙️ Secret Environment Variables — Security Patterns

```
// ── CORRECT: withCredentials — secrets masked in logs ──────────────
stage('Push Image') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId:     'docker-registry',
                usernameVariable:  'REGISTRY_USER',
                passwordVariable:  'REGISTRY_PASS'
            )
        ]) {
            sh 'echo $REGISTRY_PASS | docker login registry.io -u
$REGISTRY_USER --password-stdin'
            // Log shows: echo ****  (password masked)
        }
        // REGISTRY_USER and REGISTRY_PASS are UNSET after withCredentials
block
    }
}

// ── WRONG: Don't expose secrets as regular env vars ────────────────
environment {
    MY_SECRET = 'plain-text-secret'  // ❌ Visible in pipeline config,
logs
}
```

```
// ── WRONG: Don't echo secrets ──────────────────────────────────
withCredentials([string(credentialsId: 'my-token', variable: 'TOKEN')]) {
    echo "Token: ${TOKEN}"  // ❌ Jenkins masks this, but logging it is
bad practice
    sh 'echo $TOKEN'        // ❌ Even masked in Jenkins, BAD PRACTICE
}

// ── CORRECT: Credentials in environment{} block (auto-masked) ────
environment {
    SONAR_TOKEN = credentials('sonarqube-token')
    // Value masked in all log outputs ✅
    // Do NOT echo it, but it IS safe to use in steps
}

// ── CORRECT: File credentials ──────────────────────────────────
withCredentials([file(credentialsId: 'kubeconfig-prod', variable:
'KUBECONFIG')]) {
    sh 'kubectl get pods --namespace production'
    // KUBECONFIG env var points to a temp file with the kubeconfig
content
    // File is automatically deleted after withCredentials block
}
```

🔑 Key Concepts

Variable	Value	Notes
`BUILD_NUMBER`	`42`	Integer counter, always incrementing
`GIT_COMMIT`	`abc123def...`	Full 40-char SHA
`BRANCH_NAME`	`main`	Only in Multibranch pipelines
`CHANGE_ID`	`123`	PR number — only set in PR builds
`WORKSPACE`	`/home/jenkins/workspace/...`	Agent's working directory
`NODE_NAME`	`agent-linux-1`	Name of the current agent
`env.VAR`	Groovy access	Use `env.VAR` in Groovy, `$VAR` in shell
Pipeline-level `environment {}`	All stages	Broadest scope
Stage-level `environment {}`	That stage only	Overrides pipeline-level
`script { env.VAR = '...' }`	Subsequent stages	Dynamic assignment

💬 Short Crisp Interview Answer

> *"Jenkins provides rich built-in environment variables: `BUILD_NUMBER`, `JOB_NAME`, `BRANCH_NAME` (in Multibranch), `GIT_COMMIT`, `WORKSPACE`, `BUILD_URL`. Custom variables are defined in the `environment {}` block — at pipeline level for all stages, or at stage level to scope them to just that stage. Stage-level environment overrides pipeline-level for that stage. Dynamic variables can be set in `script {}` blocks with `env.VAR = value` and are available to subsequent stages. The critical rule for secrets: use `withCredentials {}` binding or the `credentials()` function in `environment {}` — both mask values in logs. Never hardcode secrets as plain string environment variables. The Groovy GString vs shell dollar sign distinction is also critical: `${env.VAR}` in double-quoted strings is Groovy interpolation happening before the shell executes; `\$VAR` is evaluated by the shell."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **`BRANCH_NAME` is only available in Multibranch Pipelines.** In a plain Pipeline job (not Multibranch), `env.BRANCH_NAME` is null/empty. Use `env.GIT_BRANCH` instead (which contains `origin/main`), or `sh(script: 'git rev-parse --abbrev-ref HEAD', returnStdout: true).trim()`.
- **`environment {}` expressions are evaluated at pipeline START.** `APP_VERSION = sh(script: 'cat VERSION', returnStdout: true).trim()` in the pipeline-level `environment {}` block runs once before any stage executes. If `VERSION` file doesn't exist at that point (e.g., checkout hasn't run yet), it fails. Pattern: put dynamic env var setup inside a `script {}` block after checkout.
- **Groovy strings are case-sensitive with `env.*`.** `env.branch_name` is NOT the same as `env.BRANCH_NAME`. Jenkins env vars are uppercase by convention, and the built-in ones are always uppercase.
- **`env.VAR = value` in `script {}` sets it globally for the pipeline run**, not just the current stage. This can cause unexpected cross-stage contamination. Use local Groovy `def` variables for stage-local values.
- **Shell environment and Jenkins environment are related but not identical.** All `env.*` variables are passed to shell processes as OS environment variables. But a shell `export FOO=bar` inside a `sh` step does NOT update `env.FOO` in Jenkins — the shell is a subprocess, its environment doesn't propagate back.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Credentials (3.12)	Secret env vars injected via credentials binding
Declarative Pipeline (3.2)	`environment {}` is a Declarative directive
Scripted Pipeline (3.3)	`env.VAR = value` is the Scripted equivalent
Shared Libraries (3.13)	Env vars accessible in library functions via `env.*`
Parameters (3.8)	`params.NAME` vs `env.NAME` — different namespaces

Topic 3.12 — Credentials Management in Jenkins

🔴 Advanced | Security Foundation

📌 What It Is — In Simple Terms

Credentials Management in Jenkins covers: how secrets (passwords, tokens, SSH keys, certificates) are stored securely in Jenkins, how they're injected into pipelines without appearing in logs or code, and how to integrate with external secrets managers like HashiCorp Vault.

This is the most security-critical topic in Jenkins. Mishandling credentials is the #1 source of production security incidents in CI/CD systems.

⚙️ Credential Types — Complete Reference

```
Jenkins stores credentials in JENKINS_HOME/credentials.xml,
encrypted with the secrets/master.key + secrets/hudson.util.Secret keys.

Credential Types:
  1. Username + Password    → service accounts, registry logins
  2. Secret Text (String)   → API tokens, bearer tokens
  3. Secret File            → kubeconfig files, certificates, JKS files
  4. SSH Username + Private Key → git clone, server access
  5. Certificate (PKCS#12)  → mTLS client certificates
  6. AWS Credentials        → AWS_ACCESS_KEY_ID + AWS_SECRET_ACCESS_KEY
  7. Kubernetes Service Account → K8s API access
  8. GitHub App             → GitHub App installation token (auto-
rotating)

Credential Scopes:
  System:  Only accessible to Jenkins system (not pipelines) — agent
connections, email
  Global:  Accessible to any pipeline
  Folder:  Accessible only to pipelines within a folder — multi-team
isolation
```

⚙️ Injecting Credentials — All Patterns

```
// ── PATTERN 1: environment{} block with credentials() ──────────────
// Cleanest for values used throughout a stage or pipeline
pipeline {
    agent any
    environment {
        // Username + Password credential:
        DOCKER_CREDS   = credentials('docker-registry')
```

```
        // Creates: DOCKER_CREDS_USR, DOCKER_CREDS_PSW, DOCKER_CREDS
(user:pass)

        // Secret Text (API token):
        GITHUB_TOKEN   = credentials('github-pat')
        // Creates: GITHUB_TOKEN (the token value, masked in logs)

        // Secret File:
        KUBECONFIG     = credentials('kubeconfig-staging')
        // Creates: KUBECONFIG (path to temp file with secret content)

        // SSH key:
        SSH_KEY        = credentials('ssh-deploy-key')
        // Creates: SSH_KEY (path to temp private key file), SSH_KEY_USR
(username)
    }
    stages {
        stage('Build') {
            steps {
                sh 'echo $DOCKER_CREDS_PSW | docker login -u
$DOCKER_CREDS_USR --password-stdin'
                // Password masked in logs: echo ****
            }
        }
    }
}

// ── PATTERN 2: withCredentials{} block — tightest scope ──────────
// Best for one-time use or when scope matters
stage('Deploy') {
    steps {
        withCredentials([
            // Username + Password:
            usernamePassword(
                credentialsId:     'docker-registry',
                usernameVariable:  'DOCKER_USER',
                passwordVariable:  'DOCKER_PASS'
            ),
            // Secret Text:
            string(
                credentialsId: 'slack-webhook-url',
                variable:      'SLACK_URL'
            ),
            // Secret File:
            file(
                credentialsId: 'kubeconfig-prod',
                variable:      'KUBECONFIG'
            ),
            // SSH Private Key:
            sshUserPrivateKey(
                credentialsId:   'deploy-ssh-key',
                keyFileVariable: 'SSH_KEY_FILE',
                usernameVariable: 'SSH_USER',
                passphraseVariable: 'SSH_PASSPHRASE'  // if key has
```

```
passphrase
            ),
            // AWS Credentials (requires AWS Credentials plugin):
            [
                $class: 'AmazonWebServicesCredentialsBinding',
                credentialsId: 'aws-production',
                accessKeyVariable: 'AWS_ACCESS_KEY_ID',
                secretKeyVariable: 'AWS_SECRET_ACCESS_KEY'
            ]
        ]) {
            sh """
                echo $DOCKER_PASS | docker login -u $DOCKER_USER --
password-stdin
                aws ecr get-login-password | docker login --username AWS -
-password-stdin 123456789.dkr.ecr.us-east-1.amazonaws.com
                kubectl --kubeconfig=$KUBECONFIG apply -f k8s/
                ssh -i $SSH_KEY_FILE $SSH_USER@deploy.example.com 'sudo
systemctl restart myapp'
            """
        }
        // All secrets are UNSET after this block — not available in
subsequent steps
    }
}

// ── PATTERN 3: Git credentials for checkout ──────────────────────
stage('Checkout') {
    steps {
        checkout([
            $class: 'GitSCM',
            branches: [[name: '*/main']],
            userRemoteConfigs: [[
                url:           'https://github.com/myorg/myapp.git',
                credentialsId: 'github-credentials'  // Username+Password
or SSH
            ]]
        ])
        // OR for SSH:
        // git url: 'git@github.com:myorg/myapp.git', credentialsId: 'ssh-
github-key'
    }
}

// ── PATTERN 4: HTTP API call with token ──────────────────────────
stage('Trigger External System') {
    steps {
        withCredentials([string(credentialsId: 'api-token', variable:
'TOKEN')]) {
            sh """
                curl -X POST https://api.example.com/deploy \
                    -H "Authorization: Bearer \$TOKEN" \
                    -H "Content-Type: application/json" \
                    -d '{"version": "${env.BUILD_NUMBER}"}'
            """
```

```
            // \$TOKEN — shell variable (Jenkins has already injected it
as env var)
            // "${env.BUILD_NUMBER}" — Groovy interpolation (safe, not a
secret)
        }
    }
}
```

⚙️ HashiCorp Vault Integration

```
// ── VAULT INTEGRATION: Dynamic, short-lived secrets ────────────────
// Requires: HashiCorp Vault Plugin

// Option 1: Vault plugin annotation approach
@Library('jenkins-shared-library') _

pipeline {
    agent any

    environment {
        // Fetch secret from Vault at pipeline start
        // vault plugin authenticates using Jenkins' AppRole or K8s SA
        VAULT_TOKEN = vaultString(
            path:    'secret/myteam/database',
            key:     'password',
            vaultUrl: 'https://vault.example.com'
        )
    }
    stages {
        stage('Deploy') {
            steps {
                sh 'psql postgresql://user:$VAULT_TOKEN@db.internal/myapp
-c "SELECT 1"'
            }
        }
    }
}

// Option 2: withVault{} step
stage('Database Migration') {
    steps {
        withVault(
            configuration: [
                vaultUrl:               'https://vault.example.com',
                vaultCredentialId:      'vault-approle-credentials',   //
Jenkins cred with AppRole
                engineVersion:          2,
                timeout:                60
            ],
            vaultSecrets: [
```

```
                [
                    path:               'secret/myteam/database',
                    engineVersion:      2,
                    secretValues: [
                        [vaultKey: 'host',     envVar: 'DB_HOST'],
                        [vaultKey: 'username', envVar: 'DB_USER'],
                        [vaultKey: 'password', envVar: 'DB_PASS']
                    ]
                ],
                [
                    path:               'secret/myteam/api',
                    secretValues: [
                        [vaultKey: 'key', envVar: 'API_KEY']
                    ]
                ]
            ]
        ) {
            sh '''
                flyway -url=jdbc:postgresql://$DB_HOST/mydb \
                       -user=$DB_USER -password=$DB_PASS migrate
            '''
        }
    }
}

// Option 3: Vault agent sidecar (Kubernetes) — secrets injected as files
// Vault Kubernetes injector automatically mounts secrets into pod
filesystem
// No Jenkins plugin needed — Vault handles it at K8s level
// Annotation on pod:
// vault.hashicorp.com/agent-inject: "true"
// vault.hashicorp.com/role: "jenkins-agent"
// vault.hashicorp.com/agent-inject-secret-db: "secret/myteam/database"
// Secrets available at: /vault/secrets/db

pipeline {
    agent {
        kubernetes {
            yaml '''
apiVersion: v1
kind: Pod
metadata:
  annotations:
    vault.hashicorp.com/agent-inject: "true"
    vault.hashicorp.com/role: "jenkins-agent"
    vault.hashicorp.com/agent-inject-secret-database:
"secret/myteam/database"
    vault.hashicorp.com/agent-inject-template-database: |
      {{- with secret "secret/myteam/database" -}}
      export DB_HOST="{{ .Data.data.host }}"
      export DB_USER="{{ .Data.data.username }}"
      export DB_PASS="{{ .Data.data.password }}"
      {{- end }}
spec:
```

```
  containers:
  - name: jnlp
    image: jenkins/inbound-agent:latest
'''
        }
    }
    stages {
        stage('DB Migration') {
            steps {
                sh '''
                    source /vault/secrets/database
                    flyway -url=jdbc:postgresql://$DB_HOST/mydb migrate
                '''
            }
        }
    }
}
```

⚙️ Credential Best Practices

```
1. SCOPE CREDENTIALS to the narrowest scope:
   - Use Folder-scoped credentials for team isolation
   - Not everything needs Global scope
   - Teams shouldn't be able to use each other's production credentials

2. NEVER IN CODE:
   - No credentials in Jenkinsfile, even in comments
   - No credentials in build parameters (except password params for
dynamic cases)
   - No credentials in environment variables defined as plain strings
   - CI system: scan Jenkinsfiles for secret patterns with gitleaks

3. CREDENTIAL ROTATION:
   - Store credentials in Jenkins with IDs (credentialsId: 'github-token')
   - Rotate the VALUE in Jenkins credentials store
   - Pipelines referencing by ID automatically get the new value
   - No Jenkinsfile changes required for rotation ✅

4. PREFER VAULT over Jenkins built-in credentials at scale:
   - Jenkins built-in: credentials encrypted with master.key (JENKINS_HOME
security risk)
   - Vault: dedicated secrets management, audit trail, dynamic secrets,
lease expiry
   - Vault dynamic database credentials: unique credentials per build,
auto-expire

5. AUDIT LOGGING:
   - Enable Audit Trail plugin in Jenkins
   - Every credential access is logged: who, which credential, when, from
which job
```

```
    - Required for SOC2/ISO27001 compliance

  6. DETECT CREDENTIAL LEAKAGE:
     - Jenkins automatically masks credential values in console output
     - But: writing to a file and archiving it exposes the secret
     - Scan archived artifacts for secrets before publishing
     - Use 'credentials-binding' plugin features: it only masks known
  patterns
       A partial match (first 3 chars of token) might not be masked
```

🔑 Key Concepts

Concept	Meaning
Credential ID	The string name used to reference a credential in pipelines
Credential scope	System / Global / Folder — controls which jobs can access it
`credentials('id')`	Injects credential into `environment {}` block (masked)
`withCredentials([...])`	Block-scoped credential injection — unset after block
Credential masking	Jenkins replaces the secret value with `****` in console logs
Dynamic secrets	Vault-generated credentials that expire after a build — most secure
AppRole auth	Vault authentication method for CI/CD — Jenkins uses AppRole to get secrets
JCasC credentials	Define credentials in JCasC YAML (values from env vars, not plaintext)

💬 Short Crisp Interview Answer

> *"Jenkins credentials are stored encrypted in JENKINS_HOME, keyed by a credential ID. Types: username+password, secret text (API tokens), secret file (kubeconfig, certs), SSH key, AWS credentials. Pipelines reference credentials by ID, never embedding the value in code. Two injection patterns: `credentials('id')` in the `environment {}` block — available as env vars throughout the stage/pipeline, masked in logs; and `withCredentials([...]) {}` — tightest scope, credentials unset after the block closes. For production at scale, I prefer HashiCorp Vault: the Vault plugin fetches dynamic secrets at build time, with full audit trail and credential rotation without Jenkinsfile changes. The critical security rules: never echo credentials even when masked (masked in Jenkins logs, but leaks if pushed to external log systems), scope credentials to the narrowest folder, and use credential IDs so rotating the value in Jenkins propagates automatically to all pipelines."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **Credential masking is bypass-able.** Jenkins masks the EXACT string value of the credential. If you `base64` encode a token and print it, Jenkins won't mask the base64 version — it doesn't know

the encoded form. Also: writing a credential to a file and archiving that file exports the secret unmasked.

- **`credentials('id')` for SSH key creates a FILE path, not the key content.** `SSH_KEY = credentials('my-ssh-key')` gives `SSH_KEY` pointing to a temp file containing the private key, NOT the key content as a string. Use `$SSH_KEY` as the path: `ssh -i $SSH_KEY user@server`.
- **Username+Password credential creates THREE variables.** `CREDS = credentials('my-creds')` creates `CREDS_USR` (username), `CREDS_PSW` (password), AND `CREDS` (username:password combined). Using `$CREDS` directly in `docker login --password` would provide `user:pass` which is wrong.
- **Credentials in pipeline logs via `sh` echo.** `sh "curl -H 'Auth: $TOKEN'"` with Groovy GString: TOKEN is interpolated at Groovy level before Jenkins can mask it. Use shell variable: `sh 'curl -H "Auth: $TOKEN"'` where TOKEN is set via `withCredentials` — Jenkins masks the shell-level value.
- **Folder-scoped credentials not visible to parent folder pipelines.** A credential scoped to folder `team-a` is NOT accessible to a pipeline at the root level or in `team-b`. Credential lookup goes: current folder → parent folder → root. Not cross-sibling.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Jenkins Home Directory (2.6)	`credentials.xml` and `secrets/` live in JENKINS_HOME
Environment Variables (3.11)	Credentials injected as environment variables
Jenkins HA/DR (2.7)	`secrets/` backup critical for credential recovery
JCasC (5.7)	Credentials defined as code with env var placeholders
Vault Integration (6.5)	Full Vault plugin and dynamic credentials

---

Topic 3.13 — Pipeline Shared Libraries ⚠

🔴 Advanced | The DRY Principle for Pipelines

📌 What It Is — In Simple Terms

Shared Libraries are reusable Groovy code that can be called from any Jenkinsfile across your organization. Instead of copy-pasting the same pipeline logic into 50 different Jenkinsfiles, you extract common logic into a library — `buildDockerImage()`, `deployToKubernetes()`, `runSonarQube()` — and every team calls these functions.

This is the single most powerful feature for pipeline maintainability at scale. It's heavily tested because it reveals whether a candidate has operated Jenkins for a real organization (not just a single project).

```
Without Shared Libraries:                With Shared Libraries:
  myapp/Jenkinsfile:                        myapp/Jenkinsfile:
    sh 'mvn clean package'                    @Library('company-lib@v1.2')
_
    sh 'docker build ...'                     pipeline {
    sh 'docker push ...'                        stages {
    sh 'helm upgrade ...'                         stage('Build') {
    sh 'curl slack_webhook ...'                     steps { buildAndPush()
}
    // 200 lines                                  }
                                                  stage('Deploy') {
  authservice/Jenkinsfile:                          steps {
deployToK8s('staging') }
    sh 'mvn clean package'                        }
    sh 'docker build ...'     ← COPY           }
    // Same 200 lines                         }
                                             // 20 lines — everything
reused
```

⚙️ Shared Library Structure — The Full Directory Layout

```
jenkins-shared-library/              ← Git repository root
│
├── vars/                            ← GLOBAL VARIABLES (pipeline steps)
│   ├── buildDockerImage.groovy      ← Call as: buildDockerImage(args)
│   ├── deployToKubernetes.groovy    ← Call as: deployToKubernetes(args)
│   ├── runSonarAnalysis.groovy      ← Call as: runSonarAnalysis()
│   ├── notifySlack.groovy           ← Call as: notifySlack(channel,
message)
│   ├── standardPipeline.groovy      ← Call as: standardPipeline { ... }
│   │
│   └── buildDockerImage.txt         ← Optional: documentation (shown in
Jenkins UI)
│       (same name as .groovy, .txt extension)
│
├── src/                             ← CLASSES (Groovy source, like a
library)
│   └── com/
│       └── example/
│           └── pipeline/
│               ├── BuildUtils.groovy     ← Reusable class
│               ├── DeploymentManager.groovy
│               └── SlackNotifier.groovy
│
├── resources/                       ← STATIC RESOURCES (config files,
scripts)
│   ├── com/example/pipeline/
│   │   ├── default-values.yaml      ← Loaded with libraryResource()
│   │   └── helm-template.yaml
```

```
│   └── scripts/
│       └── health-check.sh          ← Shell scripts bundled with library
│
└── test/                            ← UNIT TESTS (JenkinsPipelineUnit)
    └── groovy/
        └── BuildDockerImageTest.groovy
```

⚙️ vars/ — Global Variables (Most Common)

```
// ── vars/buildDockerImage.groovy ──────────────────────────────────
// Call from Jenkinsfile: buildDockerImage(name: 'myapp', tag: '1.0.0')

def call(Map config = [:]) {
    // Provide defaults for optional params
    def imageName  = config.name    ?: error('image name is required')
    def imageTag   = config.tag     ?: env.BUILD_NUMBER
    def registry   = config.registry ?: 'registry.example.com'
    def dockerfile = config.dockerfile ?: 'Dockerfile'
    def context    = config.context ?: '.'
    def pushImage  = config.push    ?: true
    def buildArgs  = config.buildArgs ?: [:]

    def buildArgsStr = buildArgs.collect { k, v -> "--build-arg ${k}=${v}"
}.join(' ')
    def fullImage    = "${registry}/${imageName}:${imageTag}"

    echo "📦 Building Docker image: ${fullImage}"

    sh """
        docker build \
            -f ${dockerfile} \
            ${buildArgsStr} \
            -t ${fullImage} \
            ${context}
    """

    // Scan image for vulnerabilities
    sh "trivy image --exit-code 0 --severity HIGH,CRITICAL --format table
${fullImage}"

    if (pushImage) {
        sh """
            echo \$DOCKER_CREDS_PSW | docker login ${registry} \
                -u \$DOCKER_CREDS_USR --password-stdin
            docker push ${fullImage}
        """
        echo "✅ Pushed: ${fullImage}"
    }
```

```
    return fullImage  // Return the full image reference
}
```

```
// ── vars/deployToKubernetes.groovy ──────────────────────────────

def call(String environment, Map config = [:]) {
    def chartPath   = config.chart     ?: './chart'
    def namespace   = config.namespace ?: environment
    def releaseName = config.release   ?: env.APP_NAME ?: error('APP_NAME
not set')
    def imageTag    = config.imageTag  ?: env.BUILD_NUMBER
    def valuesFile  = config.values    ?: "values-${environment}.yaml"
    def timeout     = config.timeout   ?: '5m'

    withCredentials([file(credentialsId: "kubeconfig-${environment}",
variable: 'KUBECONFIG')]) {
        sh """
            helm upgrade --install ${releaseName} ${chartPath} \
                --namespace ${namespace} \
                --create-namespace \
                --set image.tag=${imageTag} \
                -f ${valuesFile} \
                --wait \
                --timeout=${timeout} \
                --atomic \
                --history-max=5
        """
    }
    echo "✅ Deployed ${releaseName}:${imageTag} to ${environment}"
}
```

```
// ── vars/notifySlack.groovy ─────────────────────────────────────

// Multiple signatures — call with different args:
// notifySlack('success')
// notifySlack('failure', '#specific-channel')
// notifySlack(status: 'success', channel: '#deploys', message: 'Custom
message')

def call(String status, String channel = '#builds') {
    call(status: status, channel: channel)
}

def call(Map config) {
    def status  = config.status  ?:
currentBuild.currentResult.toLowerCase()
    def channel = config.channel ?: '#builds'
    def message = config.message
```

```
    def emoji   = [success: '✅', failure: '❌', unstable: '⚠', aborted:
'🚫']
    def color   = [success: 'good', failure: 'danger', unstable:
'warning', aborted: '#808080']

    def buildInfo = "*${env.JOB_NAME}* #${env.BUILD_NUMBER}"
    def duration  = currentBuild.durationString.replace(' and counting',
'')
    def msg       = message ?: "${emoji[status] ?: '🔔'} ${buildInfo}
${status.toUpperCase()} (${duration})\n${env.BUILD_URL}"

    slackSend(
        channel: channel,
        color:   color[status] ?: '#808080',
        message: msg
    )
}
```

```
// ── vars/standardPipeline.groovy — Pipeline Template ──────────────
// This is the most powerful pattern: a complete pipeline template
// Call from Jenkinsfile as a DSL block:
//
// standardPipeline {
//     appName = 'myservice'
//     skipTests = false
//     deployToEnvs = ['staging', 'production']
// }

def call(Closure body) {
    // Execute the body closure to collect configuration
    def config = [:]
    body.resolveStrategy = Closure.DELEGATE_FIRST
    body.delegate = config
    body()

    // Provide defaults
    def appName     = config.appName     ?: error('appName required')
    def skipTests   = config.skipTests   ?: false
    def registry    = config.registry    ?: 'registry.example.com'
    def deployEnvs  = config.deployToEnvs ?: ['staging']

    // The actual pipeline — defined in the library
    pipeline {
        agent {
            kubernetes { inheritFrom 'standard-build-pod' }
        }

        options {
            buildDiscarder(logRotator(numToKeepStr: '20'))
            timeout(time: 60, unit: 'MINUTES')
            timestamps()
```

```
            disableConcurrentBuilds(abortPrevious: true)
        }

        stages {
            stage('Build & Test') {
                steps {
                    container('maven') {
                        sh skipTests ? 'mvn clean package -DskipTests' :
'mvn clean verify'
                    }
                }
                post {
                    always {
                        junit allowEmptyResults: true, testResults:
'target/surefire-reports/*.xml'
                    }
                }
            }

            stage('Build Image') {
                steps {
                    script {
                        buildDockerImage(
                            name:     appName,
                            tag:
"${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}",
                            registry: registry
                        )
                    }
                }
            }

            stage('Deploy') {
                steps {
                    script {
                        deployEnvs.each { env ->
                            if (env == 'production') {
                                input message: "Deploy to production?",
ok: 'Deploy',
                                      submitter: 'release-managers'
                            }
                            deployToKubernetes(env, [
                                release: appName,
                                imageTag:
"${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}"
                            ])
                        }
                    }
                }
            }
        }

        post {
            success { notifySlack('success') }
```

```
            failure { notifySlack('failure') }
            always  { cleanWs() }
        }
    }
}
```

⚙️ src/ — Groovy Classes

```
// ── src/com/example/pipeline/BuildUtils.groovy ──────────────────
package com.example.pipeline

class BuildUtils implements Serializable {
    // Serializable required for CPS pipeline durability

    def script  // Reference to pipeline script context (needed for sh,
echo, etc.)

    BuildUtils(script) {
        this.script = script
    }

    def getGitVersion() {
        return script.sh(
            script: 'git describe --tags --always --dirty',
            returnStdout: true
        ).trim()
    }

    def getChangedServices(String baseBranch = 'main') {
        def diff = script.sh(
            script: "git diff --name-only origin/${baseBranch}...HEAD",
            returnStdout: true
        ).trim()

        return diff.split('\n')
            .collect { it.split('/').first() }  // Get top-level directory
            .unique()
            .findAll { script.fileExists("services/${it}/pom.xml") }
    }

    @NonCPS  // This method uses non-CPS-safe Groovy - mark it
    def parseVersion(String version) {
        def matcher = version =~ /^(\d+)\.(\d+)\.(\d+)/
        if (!matcher.matches()) {
            throw new IllegalArgumentException("Invalid version:
${version}")
        }
        return [
            major: matcher[0][1].toInteger(),
            minor: matcher[0][2].toInteger(),
```

```
            patch: matcher[0][3].toInteger()
        ]
    }
}
```

```
// Using BuildUtils class in a Jenkinsfile:
@Library('jenkins-shared-library@v2.4.0') _
import com.example.pipeline.BuildUtils

pipeline {
    agent any
    stages {
        stage('Smart Build') {
            steps {
                script {
                    def utils = new BuildUtils(this)  // 'this' = pipeline
script context

                    def version  = utils.getGitVersion()
                    def services = utils.getChangedServices('main')

                    echo "Version: ${version}"
                    echo "Changed services: ${services}"

                    if (services.isEmpty()) {
                        echo "No services changed — skipping build"
                        currentBuild.result = 'NOT_BUILT'
                        return
                    }

                    services.each { svc ->
                        buildDockerImage(name: svc, tag: version)
                    }
                }
            }
        }
    }
}
```

⚙️ resources/ — Static Files

```
// ── resources/com/example/pipeline/default-helm-values.yaml ──────
# This file is bundled with the shared library

replicaCount: 1
resources:
  requests:
    cpu: "100m"
```

```
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
livenessProbe:
  path: /health/live
readinessProbe:
  path: /health/ready
```

```
// Loading resources in a vars/ function:
def call(Map config) {
    // Load default values from library resources
    def defaultValues = libraryResource('com/example/pipeline/default-
helm-values.yaml')

    // Write to workspace, merge with app-specific values
    writeFile file: 'library-defaults.yaml', text: defaultValues

    sh """
        helm upgrade --install ${config.name} ./chart \
            -f library-defaults.yaml \
            -f values-${config.env}.yaml \
            --set image.tag=${config.tag}
    """
}
```

⚙️ @Library Annotation — Loading the Library

```
// ── HOW TO LOAD A SHARED LIBRARY ──────────────────────────────

// 1. PINNED VERSION (production best practice):
@Library('jenkins-shared-library@v2.4.0') _
// Use a git tag — predictable, reproducible

// 2. BRANCH (for development of the library):
@Library('jenkins-shared-library@feature/new-deploy-step') _

// 3. SPECIFIC COMMIT SHA (most reproducible):
@Library('jenkins-shared-library@abc123def456') _

// 4. LATEST (AVOID in production — breaks unpredictably):
@Library('jenkins-shared-library') _    // Uses default branch — dangerous

// 5. MULTIPLE LIBRARIES:
@Library(['jenkins-shared-library@v2.4.0', 'security-library@v1.0.0']) _

// 6. DYNAMIC LOADING inside pipeline (less common):
library('jenkins-shared-library@v2.4.0')
```

```
// Can be called inside script{} for conditional loading

// The trailing underscore _ after @Library is REQUIRED when no import
follows
// It tells Groovy the annotation is complete (suppresses "statement
expected" error)
// If you import a class, the _ is not needed:
@Library('jenkins-shared-library@v2.4.0')
import com.example.pipeline.BuildUtils
```

⚙️ Library Configuration in Jenkins

```
# JCasC — configure shared library so it's available to all pipelines:
unclassified:
  globalLibraries:
    libraries:
    - name: "jenkins-shared-library"      # Name used in @Library
annotation
      defaultVersion: "main"              # Default branch if no version
specified
      retriever:
        modernSCM:
          scm:
            git:
              remote: "https://github.com/myorg/jenkins-shared-
library.git"
              credentialsId: "github-library-creds"
      allowVersionOverride: true           # Allow @Library('lib@specific-
version')
      includeInChangesets: false           # Don't count library changes
as job changes
      implicit: false                      # Require explicit @Library
annotation
```

🔑 Key Concepts

Concept	Meaning
`vars/`	Global step functions — `buildDockerImage()` in pipeline
`src/`	Groovy classes — complex logic, reusable objects
`resources/`	Static files — loaded via `libraryResource()`
`@Library('name@version')` `_`	Import library with version pin
`call(args)`	The `call` method is what executes when you call `buildDockerImage(args)`

Concept	Meaning
Pipeline template	`vars/` function that contains an entire `pipeline {}` block
Implicit library	Auto-loaded for all pipelines without `@Library` annotation
`Serializable`	Classes in `src/` must implement `Serializable` for CPS durability
`libraryResource()`	Load a file from `resources/` directory of the library

💬 Short Crisp Interview Answer

> *"Shared Libraries are reusable Groovy code stored in a separate Git repository, importable into any Jenkinsfile with the `@Library` annotation. The library has three directories: `vars/` for global step functions — each file becomes a callable pipeline step like `buildDockerImage()`; `src/` for Groovy classes with more complex logic, similar to a Groovy library; and `resources/` for static files loaded via `libraryResource()`. The `vars/` pattern is most common in practice — you define a `call()` method in `buildDockerImage.groovy`, and every team's Jenkinsfile calls `buildDockerImage(name: 'myapp')` instead of duplicating 40 lines of Docker build, scan, and push logic. The most powerful pattern is a pipeline template: a `standardPipeline.groovy` in `vars/` that defines an entire pipeline block, which teams invoke as a DSL. Always pin library versions with `@Library('lib@v2.4.0')` — floating `@main` causes unpredictable breaks when the library is updated."*

🔬 Deep Dive Answer

Testing Shared Libraries:

```
// Unit testing with jenkins-pipeline-unit library
// https://github.com/jenkinsci/JenkinsPipelineUnit

// test/groovy/BuildDockerImageTest.groovy
import com.lesfurets.jenkins.unit.BasePipelineTest
import org.junit.Before
import org.junit.Test
import static org.junit.Assert.*

class BuildDockerImageTest extends BasePipelineTest {

    @Before
    void setUp() {
        super.setUp()
        // Register mock steps
        helper.registerAllowedMethod('sh', [String.class], { cmd ->
            return "mock output"
        })
        helper.registerAllowedMethod('echo', [String.class], { msg -> })
        helper.registerAllowedMethod('error', [String.class], { msg ->
            throw new RuntimeException(msg)
        })
```

```
        binding.setVariable('env', [BUILD_NUMBER: '42', GIT_COMMIT:
'abc1234defghij'])
        binding.setVariable('DOCKER_CREDS_USR', 'test-user')
        binding.setVariable('DOCKER_CREDS_PSW', 'test-pass')
    }

    @Test
    void testBuildDockerImageRequiresName() {
        def script = loadScript('vars/buildDockerImage.groovy')
        try {
            script.call([:])  // No name
            fail('Expected error')
        } catch (RuntimeException e) {
            assertTrue(e.message.contains('image name is required'))
        }
    }

    @Test
    void testBuildDockerImageDefaultTag() {
        def script = loadScript('vars/buildDockerImage.groovy')
        def result = script.call(name: 'myapp', push: false)

        // Verify docker build was called with correct tag
        def shCalls = helper.callStack.findAll { it.methodName == 'sh' }
        assertTrue(shCalls.any {
it.args[0].contains('registry.example.com/myapp:42') })
    }
}
```

Library Versioning Strategy:

```
# Recommended: Semantic Versioning for shared libraries

v1.0.0  → Initial release
v1.1.0  → New features (backward compatible)
v1.2.3  → Bugfix

# In Jenkinsfile:
@Library('jenkins-shared-library@v1.2.3') _

# Upgrade process:
1. Update library - create PR
2. Test against a sample pipeline on staging Jenkins
3. Merge PR, create new tag: v1.2.4
4. Announce to teams: "v1.2.4 available with fix for X"
5. Teams update their @Library versions at their own pace

# NEVER do:
@Library('jenkins-shared-library@main') _  # Surprise breaking changes
@Library('jenkins-shared-library') _        # Uses default branch - same
problem
```

```
# Org-wide enforcement (optional):
# Use a policy-as-code check in the library that validates the calling
pipeline
# follows standards (has required stages, has correct timeout settings,
etc.)
```

🏭 Real-World Production Example

Netflix's Pipeline Library Architecture: Netflix runs hundreds of services, all using a central `netflix-pipeline-library`. The library's `vars/` contains: `netflixCI()` (standard CI), `netflixDeploy()` (deployment), `spinnaker()` (integration with Spinnaker for canary deployments), and `canaryAnalysis()` (Kayenta metric analysis wrapper). Each service's Jenkinsfile is typically 10-20 lines calling these library functions. The library itself has 200+ automated tests using `jenkins-pipeline-unit`. Library changes go through a separate PR process, with automatic minor version bumping, and teams opt in to new versions. The library team maintains a compatibility matrix showing which library version works with which Jenkins LTS.

❓ Common Interview Questions & Strong Answers

Q1: ⚠ What's the difference between `vars/` and `src/` in a Shared Library? When do you use each?

> `vars/` contains global step functions — Groovy files where each file's `call()` method becomes a globally callable pipeline step. Use `vars/` for anything you want to call directly in a Jenkinsfile as a step: `buildDockerImage()`, `runTests()`, `deployToKubernetes()`. They're the "public API" of the library. `src/` contains Groovy source classes — object-oriented code organized in packages. Use `src/` for complex logic that benefits from class structure: state management, utility classes, complex parsing logic. Classes in `src/` must implement `Serializable` for CPS pipeline durability. The key distinction: `vars/` step functions are called like pipeline steps; `src/` classes are instantiated like regular Groovy objects. In practice, most organizations have 95% of their library in `vars/` for simplicity.

Q2: How do you prevent a library update from breaking all pipelines simultaneously?

> Pin library versions with semver tags. Every Jenkinsfile specifies `@Library('lib@v2.4.0')` — not `@main`. When the library is updated: create a new tag (`v2.4.1`), announce to teams, teams update at their own pace. The library maintainer maintains backward compatibility guarantees: patch versions (2.4.x) never break existing behavior, minor versions (2.x.0) add new features, major versions (3.0.0) may have breaking changes with migration guides. For org-wide migration to a new major version, use a script that scans all Jenkinsfiles and opens PRs to update the version. Never use floating versions in production pipelines.

⚠ Tricky Edge Cases & Gotchas

- ⚠ **`vars/` functions must have a `call()` method.** Writing a Groovy file in `vars/` with any other method name doesn't create a pipeline step. The file `buildDockerImage.groovy` must have a `def call(...)` method. Other methods in the file are helpers, but the entry point is always `call()`.

- **Classes in `src/` must be `Serializable`** for CPS pipeline durability. If your class holds references to non-serializable objects (like `java.io.File`), the pipeline can't be paused and resumed. Use `@NonCPS` on methods that handle non-serializable objects.
- **`@Library` must be the FIRST thing in the Jenkinsfile** (before the `pipeline {}` block or `node {}` block). Putting it after other statements is a syntax error.
- **The trailing underscore _ after `@Library(...)` is required** when there's no `import` statement following the annotation. Without it: `Groovy syntax error: expecting } but found ':'`. With an `import` statement: `@Library('lib') import com.example.Class` — no underscore needed.
- **Library functions called from within `parallel {}` blocks run on the Controller**, not on agents. If your library function calls `sh 'some-command'`, it needs to be inside a stage that has an agent. Be mindful of where library functions run relative to agent allocation.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Declarative Pipeline (3.2)	Library functions called within Declarative steps
Scripted Pipeline (3.3)	Class-based libraries commonly used in Scripted pipelines
Credentials (3.12)	Library functions encapsulate credential binding patterns
CPS Internals (2.3)	`@NonCPS`, `Serializable` requirement comes from CPS engine
JCasC (5.7)	Libraries configured in JCasC YAML for org-wide availability

---

Topic 3.14 — Matrix Builds

🔴 Advanced | Multi-Axis Testing

📌 What It Is — In Simple Terms

Matrix Builds run the same pipeline stages across multiple combinations of parameters — automatically. Instead of manually writing parallel stages for each combination (Java 11 + Ubuntu, Java 11 + Alpine, Java 17 + Ubuntu, Java 17 + Alpine), you define the axes and Jenkins generates all combinations.

Think of it as an automatic cross-product of parallel builds.

```
MANUAL (4 stages you write):           MATRIX (1 definition, 4
combinations auto-generated):
  stage('Test Java11 Ubuntu') { ... }    matrix {
  stage('Test Java11 Alpine') { ... }      axes {
  stage('Test Java17 Ubuntu') { ... }        axis { name 'JDK';  values
'11', '17' }
  stage('Test Java17 Alpine') { ... }        axis { name 'OS';   values
'ubuntu', 'alpine' }
```

```
                                            }
                                            stages { stage('Test') { ... } }
                                        }
                                        // Auto-generates: 11×ubuntu,
11×alpine, 17×ubuntu, 17×alpine
```

⚙️ Complete Matrix Build Syntax

```
pipeline {
    agent none

    stages {

        // ── BASIC MATRIX ──────────────────────────────────────────────
        stage('Multi-Platform Tests') {
            matrix {
                // Define the axes (dimensions of the matrix)
                axes {
                    axis {
                        name   'JDK_VERSION'
                        values '11', '17', '21'        // 3 values
                    }
                    axis {
                        name   'OS'
                        values 'ubuntu', 'centos', 'alpine'   // 3 values
                    }
                }
                // Total combinations: 3 × 3 = 9 parallel builds

                // OPTIONAL: Exclude specific combinations
                excludes {
                    exclude {
                        axis {
                            name   'JDK_VERSION'
                            values '11'       // Exclude JDK 11 + CentOS
combination
                        }
                        axis {
                            name   'OS'
                            values 'centos'
                        }
                    }
                    // Actual combinations: 9 total - 1 excluded = 8
builds
                }

                // Agent for each matrix cell
                agent {
                    docker {
                        image "eclipse-temurin:${JDK_VERSION}-jdk-${OS}"
```

```
                        label 'docker'
                    }
                }

                // Options applied to EACH cell
                options {
                    timeout(time: 20, unit: 'MINUTES')
                    retry(1)
                }

                // The stages to run in EACH combination
                stages {
                    stage('Compile') {
                        steps {
                            sh """
                                java -version
                                mvn compile -B
                            """
                        }
                    }

                    stage('Test') {
                        steps {
                            sh 'mvn test -B'
                        }
                        post {
                            always {
                                junit "target/surefire-reports/*.xml"
                            }
                        }
                    }

                    stage('Package') {
                        steps {
                            sh 'mvn package -DskipTests'
                            archiveArtifacts artifacts: 'target/*.jar',
allowEmptyArchive: true
                        }
                    }
                }
            }
        }

        // ── MATRIX WITH KUBERNETES AGENTS ──────────────────────────────
        stage('Cross-Platform Integration Tests') {
            matrix {
                axes {
                    axis {
                        name   'DATABASE'
                        values 'postgresql', 'mysql', 'h2'
                    }
                    axis {
                        name   'CACHE'
                        values 'redis', 'memcached'
```

```
                    }
                }
                // 3 × 2 = 6 combinations: pg+redis, pg+memcached,
mysql+redis, etc.

                agent {
                    kubernetes {
                        yaml """
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: jnlp
    image: jenkins/inbound-agent:latest-jdk17
  - name: app
    image: eclipse-temurin:17-jdk
    command: ['cat']
    tty: true
  - name: db
    image: ${DATABASE}:latest
    env:
    - name: POSTGRES_PASSWORD
      value: test
    - name: MYSQL_ROOT_PASSWORD
      value: test
  - name: cache
    image: ${CACHE}:alpine
"""
                    }
                }

                stages {
                    stage('Integration Test') {
                        steps {
                            container('app') {
                                sh """
                                    mvn verify \
                                        -Pintegration \
                                        -
Dspring.datasource.url=jdbc:${DATABASE}://localhost/testdb \
                                        -Dspring.cache.type=${CACHE}
                                """
                            }
                        }
                        post {
                            always {
                                junit 'target/failsafe-reports/*.xml'
                            }
                        }
                    }
                }
            }
        }
```

```
        // ── MATRIX WITH WHEN CONDITIONS (selective cell execution) ─────
        stage('Environment-Specific Deploy') {
            matrix {
                axes {
                    axis {
                        name   'REGION'
                        values 'us-east-1', 'eu-west-1', 'ap-southeast-1'
                    }
                    axis {
                        name   'TIER'
                        values 'staging', 'production'
                    }
                }

                when {
                    anyOf {
                        // Always run staging for all regions
                        expression { return env.TIER == 'staging' }
                        // Only run production when on main branch
                        allOf {
                            expression { return env.TIER == 'production' }
                            branch 'main'
                        }
                    }
                }

                agent { label 'deploy' }

                stages {
                    stage('Deploy') {
                        steps {
                            sh """
                                helm upgrade --install myapp ./chart \
                                    --namespace ${TIER} \
                                    --set image.tag=${BUILD_NUMBER} \
                                    --set region=${REGION} \
                                    -f values-${TIER}-${REGION}.yaml
                            """
                        }
                    }

                    // Production matrix cells require approval
                    stage('Approve') {
                        when {
                            expression { return env.TIER == 'production' }
                        }
                        steps {
                            input(
                                message: "Approve production deploy to
${REGION}?",
                                submitter: 'sre-team'
                            )
                        }
                    }
```

```
                }
            }
        }
    }
}
```

⚙️ Matrix Environment Variables

```
// Inside matrix stages, axis values are available as environment
variables:
stage('Matrix Build') {
    matrix {
        axes {
            axis { name 'JDK_VERSION'; values '11', '17' }
            axis { name 'OS';          values 'ubuntu', 'alpine' }
        }
        stages {
            stage('Info') {
                steps {
                    // Axis values available as env vars:
                    echo "JDK Version: ${JDK_VERSION}"      // shell var
                    echo "OS: ${env.OS}"                     // Groovy env
access

                    // Use in Docker image name:
                    sh "docker run eclipse-temurin:${JDK_VERSION}-
jdk-${OS} java -version"

                    // Use in file paths:
                    sh "mvn test -Djdk.version=${JDK_VERSION} -
Dos.type=${OS}"
                }
            }
        }
    }
}
```

🔑 Key Concepts

Concept	Meaning
`matrix {}`	Block that defines axes and stages for multi-combination execution
`axes {}`	Defines the dimensions of the matrix
`axis { name; values }`	One dimension — e.g., JDK versions or OS types
`excludes {}`	Skip specific axis value combinations

Concept	Meaning
`exclude {}`	One specific combination to exclude
Cross-product	Total builds = product of all axis value counts
Matrix cell	One specific combination running as a parallel build
Axis env vars	Each axis name is available as an env var inside matrix stages
`failFast` in matrix	Same as parallel — stop all when one cell fails

---

💬 Short Crisp Interview Answer

> *"Matrix builds automatically generate parallel builds for every combination of axis values — the cross-product. You define axes like `JDK_VERSION: [11, 17, 21]` and `OS: [ubuntu, alpine]`, and Jenkins runs all 6 combinations in parallel, each getting its own agent. Axis values are available as environment variables inside the matrix stages. The `excludes {}` block lets you skip specific combinations — useful when certain combinations are known unsupported. Without matrix, you'd write each combination as a separate `parallel` stage, which is error-prone, verbose, and hard to extend. Adding a new JDK version to a matrix means adding one value to an axis; in manual parallel stages it means adding a new stage block."*

🔬 Deep Dive Answer

Matrix vs Manual Parallel — Code Comparison:

```
// ── MANUAL PARALLEL (brittle, verbose): ──────────────────────────
stage('Tests') {
    parallel {
        stage('JDK 11 + Ubuntu') {
            agent { docker { image 'eclipse-temurin:11-jdk-ubuntu' } }
            steps { sh 'mvn test' }
            post { always { junit 'target/surefire-reports/*.xml' } }
        }
        stage('JDK 11 + Alpine') {
            agent { docker { image 'eclipse-temurin:11-jdk-alpine' } }
            steps { sh 'mvn test' }
            post { always { junit 'target/surefire-reports/*.xml' } }
        }
        stage('JDK 17 + Ubuntu') {
            agent { docker { image 'eclipse-temurin:17-jdk-ubuntu' } }
            steps { sh 'mvn test' }
            post { always { junit 'target/surefire-reports/*.xml' } }
        }
        stage('JDK 17 + Alpine') {
            agent { docker { image 'eclipse-temurin:17-jdk-alpine' } }
            steps { sh 'mvn test' }
            post { always { junit 'target/surefire-reports/*.xml' } }
        }
        // Add JDK 21: must add 2 new stages — error-prone
```

```
    }
}

// ── MATRIX (clean, extensible): ──────────────────────────────────
stage('Tests') {
    matrix {
        axes {
            axis { name 'JDK'; values '11', '17', '21' }  // Add JDK 21:
add one value ✅
            axis { name 'OS';  values 'ubuntu', 'alpine' }
        }
        agent { docker { image "eclipse-temurin:${JDK}-jdk-${OS}" } }
        stages {
            stage('Test') {
                steps { sh 'mvn test' }
                post { always { junit 'target/surefire-reports/*.xml' } }
            }
        }
    }
}
```

Matrix Resource Planning:

```
Matrix cells run in PARALLEL by default (like parallel stages)
Each cell needs one executor

Example: JDK [11, 17, 21] × OS [ubuntu, alpine] = 6 cells
  Need: 6 executor slots simultaneously
  Kubernetes: 6 pods created simultaneously ✅
  Static agents: need 6 free executors across agents

Large matrices: consider if all combinations are actually needed
  5 × 5 = 25 builds running simultaneously
  Agent cost: 25 pods × resources = significant
  Time: still only as long as the longest cell

Matrix + failFast = cancel all cells when any fails (same as parallel)
```

🏭 Real-World Production Example

Apache Commons Matrix Testing: The Apache Commons project uses matrix builds to ensure compatibility. Their matrix: `JDK: [8, 11, 17, 21]` × `OS: [linux, windows, macos]` = 12 builds per commit. They exclude `JDK 8 + macOS` (known unsupported) and `JDK 8 + Windows` (no CI agents available). The 10 remaining builds run in parallel — total CI time is the longest single combination's test duration. New JDK releases are added to the matrix in a single line, instantly testing all OS combinations without touching pipeline structure.

❓ Common Interview Questions & Strong Answers

Q1: You need to test a library against 4 JDK versions and 3 OS types. How do you structure this in Jenkins?

> Matrix build. Define two axes: `JDK_VERSION: [8, 11, 17, 21]` and `OS: [ubuntu, centos, alpine]`. Total: 12 parallel builds. Each cell gets a Docker agent using `eclipse-temurin:${JDK_VERSION}-jdk-${OS}` image. Axis values become environment variables usable in the image name. If certain combinations are known problematic — say JDK 8 isn't supported on Alpine — use `excludes {}` to skip that combination. The result: 11 parallel builds running the same test suite. Adding JDK 25 in the future means adding one value to the `values` list — not adding new stage blocks.

Q2: What's the difference between matrix builds and manually written parallel stages?

> Matrix is auto-generates parallel builds from axis combinations — the cross-product. Manual parallel stages require writing each combination explicitly. The practical difference: with matrix, adding a new JDK version to a 4×3 matrix adds one string to a `values` list and adds 3 new parallel builds. With manual parallel, you write 3 new stage blocks — error-prone and tedious. Matrix also ensures all combinations are tested consistently: the same stages run for every cell, so you can't accidentally forget to add test publishing to one combination. For ad-hoc parallelism where each branch does different work, use `parallel {}`. For systematic multi-combination testing, use `matrix {}`.

⚠ Tricky Edge Cases & Gotchas

- **Matrix requires Jenkins 2.241+ and Pipeline: Declarative 1.7.0+.** Older Jenkins doesn't have `matrix {}` syntax — it fails with a cryptic parse error.
- **`excludes {}` syntax is exclusion by VALUE, not combination logic.** `exclude { axis { name 'JDK'; values '8' } axis { name 'OS'; values 'centos' } }` excludes only the specific cell JDK8+CentOS, not all JDK8 cells or all CentOS cells.
- **Matrix cells sharing state.** Matrix cells share no state — no automatic stash between cells. If one cell produces an artifact needed by a later stage outside the matrix, stash it in the matrix and unstash outside. Or use an artifact repository.
- **`failFast` in matrix stops all cells on first failure.** This is the same behavior as `failFast true` in `parallel {}`. Good for: expensive multi-platform builds where all must pass. Bad for: collecting all combination failures for a comprehensive compatibility report.
- **Large matrices create agent provisioning storms.** A 5×5=25 cell matrix requests 25 Kubernetes pods simultaneously. Cluster autoscaler may not provision them all instantly — some cells queue. This is expected behavior but plan cluster capacity for peak matrix workloads.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Parallel Stages (3.7)	Matrix generates parallel stages automatically
Agent Types (2.4)	Each matrix cell gets its own agent

Connected Topic	How It Connects
Parameters (3.8)	Matrix supersedes parameter-driven multi-axis approaches
Jenkins at Scale (2.8)	Large matrices multiply agent demand
Docker Agents (2.4)	Docker images parameterized with axis values

---

📊 Category 3 Summary — Quick Reference

Topic	Core Concept	Interview Priority
3.1 Freestyle vs Pipeline	Pipeline/Jenkinsfile always over Freestyle in new work	⭐⭐⭐
3.2 Declarative Pipeline	Full `pipeline{}` syntax with all directives — memorize structure	⭐⭐⭐⭐⭐
3.3 Scripted Pipeline	`node{}` blocks, Groovy loops, dynamic stages	⭐⭐⭐⭐
3.4 Declarative vs Scripted ⚠	When to use each, hybrid approach	⭐⭐⭐⭐⭐
3.5 Jenkinsfile	Best practices, version control, anatomy	⭐⭐⭐⭐
3.6 Stages/Steps/Post	Steps reference, post conditions — know all conditions	⭐⭐⭐⭐
3.7 Parallel Stages ⚠	`failFast`, closure capture bug, dynamic parallel	⭐⭐⭐⭐⭐
3.8 Parameters	All types, validation patterns, first-run bootstrap	⭐⭐⭐⭐
3.9 Triggers	Webhooks > polling, H syntax, upstream trigger	⭐⭐⭐⭐
3.10 Stash/Unstash ⚠	When to use vs alternatives, 100MB limit	⭐⭐⭐⭐
3.11 Environment Variables	Built-ins, scopes, GString vs shell $, secret rules	⭐⭐⭐⭐
3.12 Credentials ⚠	All binding types, Vault integration, security rules	⭐⭐⭐⭐⭐
3.13 Shared Libraries ⚠	vars/ vs src/, @Library version pin, pipeline templates	⭐⭐⭐⭐⭐
3.14 Matrix Builds	axes, excludes, cell env vars, vs manual parallel	⭐⭐⭐

🔑 The Mental Model for Category 3

```
A PIPELINE is a directed graph of STAGES
  → each stage has STEPS (the actual work)
  → stages run on AGENTS (where the work happens)
  → ENVIRONMENT VARIABLES carry context through stages
  → CREDENTIALS inject secrets safely
  → POST handles results and cleanup
```

```
DECLARATIVE gives you the FORM:  pipeline { agent / options / stages /
post }
SCRIPTED gives you the FREEDOM:  node { stage / try-catch / any Groovy }
HYBRID gives you the BEST:       Declarative structure + script{} for
complex Groovy

SHARED LIBRARIES turn repeated patterns into reusable steps
  → 50 Jenkinsfiles of 200 lines → 50 Jenkinsfiles of 15 lines
  → Same quality gates, same build process, one place to maintain

PARALLEL STAGES turn 14 minutes into 5 minutes
  → Lint + Tests + SAST + DepCheck all simultaneously
  → Remember: failFast kills all if one fails, closure capture bug

SECRETS stay secrets
  → withCredentials{} or credentials() — never plaintext
  → Masked in logs, unset after block closes
  → Rotate by changing the VALUE in Jenkins — not the Jenkinsfile
```

🧪 Self-Quiz — 15 Must-Answer Interview Questions

1. Write a Declarative Pipeline for a Java service: checkout, test, docker build, deploy to staging, manual approval, deploy to production. Include proper `post {}` blocks.
2. What's the difference between `agent none` + per-stage agents vs a global `agent`? When would you use each?
3. A `parallel {}` block has 4 branches but your agent pool only has 3 executor slots. What happens?
4. Explain the Groovy closure variable capture bug in dynamic parallel stages. Show the broken code and the fix.
5. What is `failFast true` in parallel stages and when should you NOT use it?
6. Write a `withCredentials` block that injects a username+password credential and an SSH key simultaneously.
7. What is `BRANCH_NAME` and when is it NOT available?
8. Explain the difference between `${env.VAR}` in double quotes vs `$VAR` in single quotes in a `sh` step.
9. When would you choose Scripted Pipeline over Declarative? Give a concrete use case.
10. You have 40 Jenkinsfiles all containing the same 60-line Docker build/push/scan block. How do you fix this?
11. What's the structure of a Jenkins Shared Library? What goes in `vars/` vs `src/`?
12. A team adds `parameters {}` to their Jenkinsfile. The webhook fires, a build starts, but no parameter prompts appear. Why?

13. What are the stash/unstash size limits and what do you use instead for large files?
14. How do you trigger a build with specific parameter values from an upstream pipeline?
15. Design a matrix build that tests a Python library against Python 3.9, 3.10, 3.11, and 3.12 on both Ubuntu and Alpine, excluding Python 3.9 + Alpine.

Next: Category 4 — Jenkins Pipelines Advanced Patterns (Blue-Green, multi-repo, pipeline optimization)
Or: "quiz me" to test yourself on Category 3

> **Document:** Category 3 Complete (Part 2: Topics 3.8–3.14) | DevOps/SRE Interview Prep **Themes:** The most production-relevant Jenkins knowledge — pipelines you'll write and operate daily